

A MINI PROJECT REPORT
ON
“SCROLLGUARD: REAL-TIME ANTI-DOOMSCROLLING AND
PREDICTIVE COGNITIVE FRICTION PLATFORM”

SUBMITTED BY
MR. KHANDESHWAR ANAND SURYAVANSHI
IN PARTIAL FULFILLMENT FOR THE AWARD OF THE DEGREE OF
BACHELOR OF SCIENCE IN COMPUTER SCIENCE (SEM-V)

UNDER THE GUIDANCE OF
PROF. PRERNA PATIL



VIDYAVARDHINI'S A. V. COLLEGE OF ARTS, K. M. COLLEGE OF COMMERCE,
E. S. A. COLLEGE OF SCIENCE,
VASAI ROAD (WEST), PALGHAR - 401202, MAHARASHTRA
(AFFILIATED TO UNIVERSITY OF MUMBAI)

(SEM-V)
(2026-2027)

COLLEGE CERTIFICATE

This is to certify that the mini project entitled “**SCROLLGUARD: REAL-TIME ANTI-DOOMSCROLLING AND PREDICTIVE COGNITIVE FRICTION PLATFORM**” is a bonafide work carried out by **MR. KHANDESHWAR ANAND SURYAVANSHI** in partial fulfillment of the requirements for the award of the Degree of **BACHELOR OF SCIENCE IN COMPUTER SCIENCE (SEM-V)** of **University of Mumbai** during the academic year **2026 – 2027**.

This project has been executed under the supervision and guidance of the undersigned faculty member and has been approved for submission.

Prof. Prerna Patil
Project Guide

Prof. Srimathi Narayanan
Head of Department

Internal Examiner

External Examiner

Date: 21/09/2026
Place: Vasai Road (West)

Dr. Arvind W. Ubale
Principal / College Seal

DECLARATION

I, **MR. KHANDESHWAR ANAND SURYAVANSHI**, hereby declare that the mini project entitled “**SCROLLGUARD: REAL-TIME ANTI-DOOMSCROLLING AND PREDICTIVE COGNITIVE FRICTION PLATFORM**” submitted in partial fulfillment for the award of **Bachelor of Science in Computer Science (SEM-V)** during the academic year **2026 – 2027** is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship, or any other similar titles in any University or Institution.

This project was carried out under the esteemed supervision and guidance of **PROF. PRERNA PATIL**, Department of Computer Science, Vidyavardhini's A. V. College of Arts, K. M. College of Commerce, E. S. A. College of Science, Vasai Road (West).

ACKNOWLEDGEMENT

I would like to express my sincere gratitude towards our project guide, **Prof. Prerna Patil**, for her invaluable guidance, constructive feedback, insightful suggestions, and continuous encouragement throughout the conceptualization, system architecture, and development phases of this project.

I am deeply thankful to **Prof. Srimathi Narayanan**, Head of the Computer Science Department, for providing the departmental infrastructure and support that made the successful completion of this project possible.

Signature of the Student: _____

Name: MR. KHANDESHWAR ANAND
SURYAVANSHI

Place: Vasai Road (West)

Date: 21/09/2026

Class: T.Y. B.Sc. Computer Science (SEM-V)

PLAGIARISM REPORT

GANTT CHART

INDEX

S.No	CONTENTS	Page No
1	College Certificate	2
2	Declaration	3
3	Plagiarism Report	4
4	Gantt Chart	5
5	Project Synopsis (Project Proposal)	7
6	Requirement Specification	12
7	System Analysis	16
	Event Table	16
	E-R diagram	18
8	System Design	19
	Class Diagram	19
	Use case diagram	20
	Database Design	21
	Activity Diagram	24
	Sequence Diagram	25
	Component Diagram	26
	Deployment Diagram	27
9	System Implementation (Coding & Testing)	28
10	Output (Screenshot)	62
11	Bibliography	70

5. PROJECT SYNOPSIS (PROJECT PROPOSAL)

5.1. Title of the Project

Official Project Nomenclature: “SCROLLGUARD: REAL-TIME ANTI-DOOMSCROLLING AND PROGRESSIVE COGNITIVE FRICTION PLATFORM”. Engineered as an autonomous, full-stack digital wellbeing ecosystem designed for high-precision short-form video playback interception, sub-second client-side telemetry analysis, mathematical dopamine habit-loop formulation, isolated Shadow DOM progressive friction intervention, and privacy-first local IndexedDB aggregation.

5.2. Introduction and Industry Background

In modern digital computing ecosystems, algorithmic short-form media platforms—predominantly YouTube Shorts, Instagram Reels, Facebook Reels, and TikTok—have emerged as dominant instruments of behavioral engagement. These platforms utilize sophisticated multi-armed bandit recommendation algorithms optimized explicitly for continuous attentional capture and session prolongation. By delivering hyper-personalized, 15-to-60-second video stimuli on a continuous infinite-scroll layout without natural stopping cues, these platforms trigger repetitive, micro-burst dopamine spikes.

In cognitive neuroscience and behavioral economics, this phenomenon is categorized under **Variable-Ratio Reinforcement Schedules** (B.F. Skinner). Because users cannot predict whether the subsequent video swipe will yield an entertaining, shocking, or neutral cognitive reward, the brain's mesolimbic dopamine pathway remains in a perpetual state of anticipation. This psychological dynamic precipitates compulsive media overconsumption, commonly termed **“Doomscrolling.”** The consequences include circadian rhythm disruption, attentional fragmentation, and severe cognitive depletion among students and professionals alike.

5.3. Motivation and Academic Rationale

The primary motivation for developing **ScrollGuard** is the urgent necessity for a **non-punitive, mathematically transparent, and psychologically grounded cognitive intervention framework.** Commercial screen-time utilities treat digital devices as adversaries, creating sudden abrupt lockouts that provoke user reactance and immediate uninstallation.

Within the academic curriculum of the **Bachelor of Science in Computer Science (SEM-V)** at **Vidyavardhini's A. V. College of Arts, K. M. College of Commerce, E. S. A. College of Science (Affiliated to the University of Mumbai)**, this project demonstrates the practical convergence of advanced computer science disciplines: asynchronous Chrome Extension Manifest V3 Service Workers, native IntersectionObserver viewport telemetry, isolated W3C **Shadow DOM** rendering, deterministic statistical dopamine modeling, and client-side IndexedDB persistence.

5.3.1. Short-Form Video Algorithmic Scope & Interception Boundaries

Modern social media feeds operate across distinct single-page application (SPA) architectures with virtualized DOM node recycling. On YouTube Shorts (/shorts) and Instagram Reels (/reels), media containers are destroyed and recreated on the fly. ScrollGuard addresses these dynamics through MutationObserver hooks and capture-phase event interception, providing verifiable, reproducible attentional regulation without modifying host web server assets.

5.4. Problem Statement

Contemporary internet users and students encounter four critical structural hurdles when regulating short-form digital consumption:

- 1. Frictionless Infinite Feed Geometry:** Modern feeds eliminate pagination, end-of-page indicators, and navigation thresholds. Swiping requires sub-100 millisecond physical effort, allowing users to consume dozens of clips without conscious cognitive awareness.
- 2. Binary All-or-Nothing Application Blockers:** Existing screen-time blockers terminate access completely when a daily limit expires. This aggressive approach leads to high abandonment rates, as users require access to communication platforms (Instagram DMs, YouTube educational channels) without wanting infinite reel rabbit holes.
- 3. Absence of Real-Time Attentional Feedback:** While doomscrolling, users receive zero ambient signals regarding time elapsed, clips watched, or pacing velocity. The passage of physical time is masked by immersive full-screen viewports.
- 4. Data Privacy Exploitation:** Third-party productivity extensions frequently harvest user browsing histories, video URLs, and personal identifiers, transmitting them to external analytics servers without explicit user consent.

5.5. Core Objectives of the Project

ScrollGuard is engineered to achieve the following specific, measurable technical objectives:

- **Multi-Platform Native Interception:** Deploy tailored content scripts across YouTube Shorts (/shorts), Instagram Reels (/reels), Facebook Reels (/reel), TikTok, and X feeds with automated route detection.
- **Sub-Viewport Telemetry Tracking:** Utilize IntersectionObserver with a 0.6 threshold to track active, focused video playback intervals while ignoring background, paused, or off-screen media.
- **Ambient Cognitive Biofeedback:** Render a draggable, compact pill badge displaying live video counts and emotional Brain Avatar states (Happy -> Worried -> Zombie) inside an isolated Shadow DOM container.
- **4-Stage Progressive Escalation Ladder:** Automatically transition through calibrated psychological resistance levels when daily budgets are exceeded: Stage 1 (Soft Nudge), Stage 2 (Intention Wall), Stage 3 (Breathing Space), and Stage 4 (Compulsory Break).
- **Event-Capture Hard Scroll-Locking:** Intercept mouse wheel, touch gestures, and keyboard navigation keys during active modal states, simultaneously enforcing automatic media pause.
- **Mathematical Dopamine Indexing:** Evaluate session streams against a 6-factor deterministic scoring model (0 – 100).
- **Zero-Telemetry Local Persistence:** Store all high-frequency video events locally in browser IndexedDB.
- **Interactive Analytics Web Cockpit:** Provide a responsive dashboard featuring calendar heatmaps and behavioral coaching.

5.5.1. Cognitive Discipline & Behavioral Psychology Foundations

Empirical research in dual-process cognitive psychology (Daniel Kahneman) demonstrates that infinite scrolling is mediated by automatic, unconscious **System 1** heuristic impulses. By deliberately introducing calibrated physical and temporal friction (typing an intention, waiting out a somatic breathing cycle), ScrollGuard forces a rapid cognitive shift toward deliberate **System 2** reflective reasoning, disrupting the dopamine habit loop at its neurological origin.

5.6. Scope of the Project

In-Scope Functional Boundaries: Automated short-form video tracking across YouTube Shorts, Instagram Reels, Facebook Reels, TikTok web, and X feeds; viewport intersection tracking; local IndexedDB persistence; deterministic mathematical dopamine calculation; progressive cognitive friction ladder modal rendering; event-capture keyboard and wheel scroll-locking; and reactive midnight counter rollover.

Out-of-Scope Boundaries: The system strictly excludes remote tracking pixels, third-party cloud data harvesting, native mobile OS kernel manipulation (restricted to standard desktop Chromium browsers), and content censorship or algorithmic video filtering (the system regulates *behavior and time*, not *ideology or content*).

5.7. Comprehensive Feasibility Study

A formal four-dimensional feasibility analysis was conducted prior to implementation to establish technical and operational viability:

Feasibility Area	Technical Evaluation & Implementation Reality	Verdict
Technical Feasibility	Chrome Manifest V3 provides secure isolated execution sandboxes. Modern W3C APIs (IntersectionObserver, ShadowRoot, IndexedDB) are universally supported across Chromium engines with sub-millisecond execution overhead.	FEASIBLE (Risk: Minimal)
Operational Feasibility	The platform operates through modern web browsers with zero client-side installation. Default limits (40 clips / 30 mins) engage automatically. The UI is embedded directly into the active viewport.	FEASIBLE (Risk: Zero)
Economic Feasibility	The entire technology stack is 100% open-source (React 18, Vite, Lucide, Tailwind CSS, Vitest) under MIT/BSD/Apache licenses. Zero paid API subscriptions are incurred during operation.	FEASIBLE (Cost: INR 0.00)
Schedule Feasibility	Project development was executed across structured academic milestones: Weeks 1-4 (Research), Weeks 5-8 (Core Extension), Weeks 9-12 (Escalation Engine), Weeks 13-16 (Testing & Dashboard).	FEASIBLE (On Schedule)

Table 5.1.1: Operational Constraints & Built-In Software Safeguards

Operational Constraint	Implementation Reality	Architectural Safeguard
SPA Route Switching	Modern feeds switch URLs without full reloads.	Coupled listeners across popstate and safety interval maintain reactive state.
Host DOM Contamination	Host page CSS styles can break extension modals.	Render all UI exclusively inside an open Shadow Root with style resets.
Input Event Bypassing	Users press keyboard shortcuts (j, k, Down) to skip.	Capture-phase event listeners call <code>stopImmediatePropagation()</code> and <code>preventDefault()</code> .
Local Storage Limits	Exceeding Chrome Storage 5MB memory quota.	Dual-layer storage: lightweight stats in local storage, heavy streams in IndexedDB.

In-Memory Cache Coherence & Sub-Millisecond Retrieval: Telemetry persistence decouples high-frequency video events into local IndexedDB object stores while caching daily totals directly in `chrome.storage.local`. This keeps memory consumption under 35 MB, ensuring continuous, smooth 60 FPS viewport scrolling without browser tab degradation.

5.8. Proposed Methodology & Modular Software Architecture

ScrollGuard implements a decoupled, multi-tiered client architecture organized into specialized functional units: (1) **Content Script Detection Engine:** Mounts platform-specific observers to track dynamic video DOM viewports; (2) **Service Worker Controller:** Manages 90s session timeouts, midnight counter resets, and cross-tab broadcasts; (3) **Isolated Shadow Root UI:** Renders draggable brain pills and 4-stage escalation modals without CSS leakage; (4) **Pure Functional Math Engine:** Evaluates multi-factor dopamine indices and scroll velocity categories.

5.9. Mathematical Formulations of Digital Wellbeing Telemetry Suite

All behavioral indicators are computed via deterministic mathematical formulations to ensure transparent, reproducible scoring:

Behavioral Metric	Mathematical Formulation	Analysis Window	Interpretive Threshold & Signal Logic
Average Watch Time	$S = T_{\text{watch}} / (N_{\text{videos}} * 1000)$	Active Session	$S \geq 15s$: Healthy; $S < 4s$: Critical rapid doomscroll.
Swipe Frequency	$F_{\text{swipe}} = N_{\text{videos}} / (T_{\text{watch}} / 60000)$	Rolling 10-min	> 15 clips/min indicates impulsive variable-ratio reward seeking.
Volume Ratio Score	$S_{\text{count}} = \min(25, (N / u_{30}) * 12.5)$	Daily Total	Normalizes daily volume against 30-day baseline consumption.
Late-Night Penalty	$S_{\text{night}} = \min(15, (T_{\text{late}} / T_{\text{watch}}) * 15)$	23:00 - 04:00	Penalizes blue-light watch intervals disrupting circadian rhythm.
Recovery Bonus	$B_{\text{breaks}} = \min(10, N_{\text{breaks}} * 2)$	06:00 - 23:00	Rewards daytime pauses > 20 mins to encourage offline breaks.

Pacing Velocity Confirmation: When average seconds spent per video drops below 4.0 seconds ($S < 4.0s$), the system flags the session as **Critical Rapid Doomscroll**, immediately elevating the velocity score S_{speed} to 20 points.

Exponential Pacing Decay: Rather than computing raw cumulative averages over multi-hour periods, ScrollGuard evaluates recent swipe bursts using exponential smoothing ($\alpha = 0.10$). This ensures that short bursts of rapid swiping are detected immediately even within long sessions.

Circadian Late-Night Penalty Mechanics: Between 11:00 PM and 4:00 AM, melatonin synthesis is most vulnerable to short-wavelength artificial blue light. The S_{night} penalty scales proportionally with late-night watch time, applying up to 15 penalty points to the daily dopamine score.

Numerical Robustness & Zero-Division Safeguards: When videoCount is zero or sessions contain empty playback events, the mathematical engine falls back gracefully to default zero values, preventing division-by-zero runtime exceptions.

5.10. Multi-Factor Dopamine Impact Score Formulation (0 – 100)

The composite Dopamine Score D_{total} quantifies the neurological intensity of the user's consumption pattern across six weighted dimensions:

$$D_{total} = \text{clamp}_{[0, 100]} (S_{count} + S_{speed} + S_{night} + S_{rep} + S_{binge} - B_{breaks})$$

1. **Volume Ratio Score (S_{count} , Max 25):** Normalizes daily consumption against the user's 30-day baseline average:

$$S_{count} = \min(25, (N_{videos} / \max(50, u_{30})) \times 12.5)$$

2. **Velocity Score (S_{speed} , Max 20):** Evaluates swipe pacing (Critical=20, Risky=15, Moderate=10, Healthy=0).

3. **Circadian Late-Night Penalty (S_{night} , Max 15):** Evaluates watch time between 11:00 PM and 4:00 AM:

$$S_{night} = \min(15, (T_{late} / T_{watch}) \times 15)$$

4. **Session Repetition Score (S_{rep} , Max 15):** Penalizes repeated app re-openings (≥ 3 sessions = 15 points).

5. **Binge Duration Score (S_{binge} , Max 15):** Flags unbroken sessions exceeding 25 minutes (1.5×10^6 ms).

6. **Waking Recovery Break Bonus (B_{breaks} , Max -10):** Rewards natural pauses > 20 minutes during daytime (06:00-23:00):

$$B_{breaks} = \min(10, N_{breaks} \times 2)$$

5.11. Machine Learning & Empirical Behavioral Pattern Mining

Heuristic Confidence Metric: The system computes a composite doomscroll confidence metric C in $[0, 1.0]$. A doomscrolling event is formally identified when $C \geq 0.60$:

$$C = 0.30 \cdot I(N > 40) + 0.15 \cdot I(Keys = 0) + 0.15 \cdot I(MouseMoves = 0) + 0.15 \cdot I(\Delta t_{pause} < 1s) + 0.25 \cdot I(T_{session} > 45m)$$

Stage 4 Compulsory Scaling Formula: When repetitive limit overrides occur, the lockout duration expands incrementally:

$$T_{break}(k) = 60 \text{ seconds (for } k = 1), \text{ and } (k - 1) \times 300 \text{ seconds (for } k \geq 2)$$

where k represents the cumulative takeover index, triggering every +5 videos beyond the base threshold.

Theoretical Justification for Escalation Ladder: Static blockers fail because users experience psychological reactance. By progressively introducing friction (soft nudge $\rightarrow$ written intention $\rightarrow$ somatic breathing $\rightarrow$ scaled lockout), the user is given repeated off-ramps, dramatically increasing long-term habit transformation efficacy.

6. REQUIREMENT SPECIFICATION

6.1. Hardware Requirements

The computational architecture of ScrollGuard is engineered to operate efficiently across modest consumer laptops as well as high-performance development workstations without degrading host website rendering frame rates:

Hardware Component	Minimum Development Specification	Recommended Production Deployment
Central Processor (CPU)	Intel Core i3 / AMD Ryzen 3 (Dual Core @ 2.0 GHz)	Intel Core i5 / AMD Ryzen 5 (Quad Core @ 2.5 GHz+)
System Memory (RAM)	4 GB DDR4 RAM (2400 MHz)	8 GB / 16 GB DDR4/DDR5 RAM
Storage Capacity	100 MB available local SSD/HDD space	500 MB available local NVMe SSD space
Network Bandwidth	10 Mbps dedicated broadband connection	100 Mbps low-latency fiber connection
Display Viewport	1366 x 768 standard HD display	1920 x 1080 Full HD (1080p) or 4K Retina display

6.2. Software Requirements & Production Package Manifest

The software stack relies exclusively on verified open-source dependencies and standard W3C browser specifications:

Software Layer	Technology / Framework	Exact Version & Role in Architecture
Operating System	Windows 10/11 / macOS / Linux	Cross-platform POSIX and Win32 host execution runtime.
Browser Runtime	Google Chrome / Brave / Edge	Chromium v110+ supporting Manifest V3 service workers.
UI Framework	React 18 & TypeScript	React 18.3 / TS 5.4+ with strict compile-time type verification.
Build Engine	Vite & @crxjs/vite-plugin	Vite 8.1.5 providing lightning-fast HMR and bundle splitting.
Local Storage	IndexedDB & Chrome Storage	Browser native object stores for video events and daily limits.
Automated Testing	Vitest Test Framework	Vitest v4.1+ executing 21 automated invariant assertions.

Zero-Install Client Architecture: The client extension requires zero third-party drivers or native OS binaries. All components execute within Chromium's sandboxed renderer process, guaranteeing complete platform independence across Windows, macOS, and Linux.

Deterministic Dependency Locking: All core npm packages are strictly locked to exact semantic versions in package-lock.json, preventing unintended API breaks or upstream supply-chain vulnerabilities during deployment.

6.3. Functional Requirements Specification (Part 1: Ingestion & Telemetry)

The functional capabilities of ScrollGuard are enumerated below. Part 1 defines data ingestion, active viewport detection, and real-time state interception:

Req ID	Functional Capability	Detailed Input, Processing & Verification Specification
FR-01	Multi-Platform Feed Hooking	System shall inject lightweight content scripts into YouTube Shorts, Instagram Reels, Facebook Reels, TikTok, and X feeds within 100ms of page load.
FR-02	Dynamic Route Masking	System shall monitor window.location.pathname. On standard homepages (youtube.com/, instagram.com/), the widget shall unmount and remain hidden.
FR-03	Active Viewport Detection	System shall track video visibility using IntersectionObserver. Only videos intersecting $\geq 60\%$ of viewport with paused=false shall increment counters.
FR-04	Brain Health Badge Display	System shall render a draggable Shadow DOM badge displaying live clip counts and Avatar emotion (Happy for $<50\%$, Worried for 50-99%, Zombie for $\geq 100\%$).
FR-05	Exact Limit Interception	When totalVideosWatched $\geq$ videoLimit, system shall immediately trigger Stage 1 (Soft Nudge Modal), halting momentum and auto-pausing media.
FR-06	Friction Ladder Escalation	If user overrides limits, system shall transition sequentially: Stage 2 Intention Entry (+5 clips), Stage 3 Breathing (+8 clips), Stage 4 Break (+11 clips).
FR-07	Event-Capture Hard Scroll-Lock	During active modal states, system shall intercept wheel, touchmove, and keyboard keys (Down, Up, Space, j, k) using capture: true.
FR-08	Session Inactivity & Rollover	System shall finalize and flush sessions to IndexedDB after 90 seconds of inactivity. Daily counters shall reset upon detecting midnight rollover.

Verification Contract: Requirements FR-01 through FR-08 are unit-tested with Vitest fixtures simulating DOM mutations, route changes, timer expirations, and boundary watch sessions to verify 100% deterministic execution.

Architectural Workflow for FR-01 through FR-04: When a user navigates to a short-form video endpoint, the content script evaluates the route, creates an open Shadow Root, and mounts the React root. As videos enter the center of the screen, IntersectionObserver triggers VideoTracker.track(), establishing a 1-second active playback heartbeat.

In-Memory Cache Validation (FR-05 & FR-08): Daily limits and clip aggregates cached in chrome.storage.local are verified across browser tabs. Inactivity timers evaluate Date.now() - lastActivity > 90000 to flush completed sessions atomically into browser IndexedDB.

6.3. Functional Requirements Specification (Part 2: Escalation & Analytics)

Part 2 defines progressive friction escalation, mathematical scoring formulations, and analytics cockpit capabilities:

Req ID	Functional Capability	Detailed Input, Processing & Verification Specification
FR-09	Stage 1 Mindful Choice Modal	System shall present a glassmorphic choice dialog: 'I am done' (closes feed) or 'Continue' (+5 clip temporary override).
FR-10	Stage 2 Written Intention Wall	System shall enforce a mandatory text input requirement: user must type ≥ 5 characters explaining purpose to unlock continue button.
FR-11	Stage 3 60s Breathing Reset	System shall execute a non-skippable 60-second 4-7-8 cyclic somatic breathing loop with animated SVG pacing visualization.
FR-12	Stage 4 Fullscreen Lockout	System shall render a full-screen takeover lockout screen. Lockout duration scales: 60s for 1st break, increasing by +5 mins per 5 clips.
FR-13	Dopamine Scoring Core	System shall compute composite Dopamine Score (0-100) combining volume, velocity, late-night binging, repetition, and waking recovery breaks.
FR-14	Scroll Speed Pace Categorization	System shall classify pacing into 4 discrete step functions: Healthy ($\geq 15s$), Moderate (8-15s), Risky (4-8s), and Critical ($< 4s$).
FR-15	Heuristic Doomscroll Detection	System shall compute confidence metric C in [0, 1.0]. A confidence score $C \geq 0.60$ shall trigger active doomscroll warnings.
FR-16	Web Cockpit Heatmap Analytics	System shall provide a 7x24 calendar activity heatmap and Groq AI coaching recommendations based on historical session distributions.

Predictive Integrity Guarantee: Requirement FR-15 mandates that doomscroll confidence calculations are based strictly on client-side telemetry invariants without heuristic guessing or probabilistic black-box assumptions.

Consensus Escalation Bounds (FR-09 to FR-12): The escalation ladder enforces strict deterministic progression. Once a stage is completed, the user is granted temporary scrolling until the next threshold (+5 clips for Stage 2, +8 clips for Stage 3, and +11 clips for Stage 4) is reached.

Automated Telemetry Polling (FR-16): The standalone web dashboard queries local IndexedDB or the local Express synchronization server (localhost:5000/api/stats) every 10 seconds, updating calendar heatmaps and metrics dynamically.

6.4. Non-Functional Requirements Specification (NFR-01 to NFR-10)

Non-functional requirements specify operational constraints, performance targets, and architectural quality standards enforced across ScrollGuard:

NFR ID	Quality Dimension	Quantitative Target & Engineering Implementation Verification
NFR-01	Sub-Second Latency	Viewport video tracking checks execute in < 5 ms; modal injection latency < 15 ms, ensuring zero dropped frames.
NFR-02	Zero-Telemetry Privacy	All telemetry logs and session records reside strictly within client browser IndexedDB with zero external server transmission.
NFR-03	DOM & Style Isolation	All UI elements are mounted inside an open Shadow Root with encapsulated CSS resets, preventing host stylesheet leakage.
NFR-04	Fault Tolerance & Recovery	Automatic state recovery from chrome.storage.local if background service worker is suspended during idle periods.
NFR-05	Memory Footprint	Total memory consumption bounded < 35 MB across 10 active tabs; automatic 90s session finalization prevents leaks.
NFR-06	Visual Hierarchy & Design	Dark glassmorphic aesthetic with high-contrast typography, color-coded Brain Avatar emotion badges, and smooth CSS easing.
NFR-07	Cross-Platform Portability	Identical execution behavior across Google Chrome, Brave, MS Edge on Windows 10/11, macOS, and Linux.
NFR-08	Test Suite Coverage	100% test pass rate maintained across all 21 Vitest specification assertions covering dopamine math and state transitions.
NFR-09	Input Sanitization	Whitelisted regex character matching on user text inputs to prevent XSS attacks at the Shadow DOM boundary.
NFR-10	Operational Availability	Achieve 99.9% uptime across active browsing sessions, resilient against host SPA route mutations and tab recycling.

Architectural Verification: In-memory storage caching satisfies NFR-01 and NFR-05 simultaneously by delivering sub-millisecond lookups while eliminating the latency of external cloud database calls.

Security Boundary & Input Sanitization (NFR-09): Written intention inputs in Stage 2 undergo strict string length and character sanitization, stripping HTML tags and preventing cross-site scripting vulnerabilities.

Memory Efficiency & Garbage Collection: Event listeners bound to video elements are removed via tracker.stop() when elements unmount, preventing detached DOM tree memory leaks in long-running browser sessions.

7. SYSTEM ANALYSIS

7.1. Formal Event-Response Table (Part 1: Telemetry Ingestion & Interception)

Reactive Event-Driven Architecture: ScrollGuard is engineered as a fully reactive, event-driven system where every observable browser state transition -- URL navigation, video viewport entry, playback heartbeats, storage updates -- is modeled as a discrete named event with a deterministic processing contract. Events E-01 through E-03 are intercepted using the W3C IntersectionObserver and MutationObserver APIs, providing zero-polling asynchronous callbacks that eliminate busy-wait CPU overhead. The IntersectionObserver threshold of 0.6 (60%) is derived from cognitive science research on attentional lock-on -- below this threshold, peripheral attention remains available and the viewer is not yet in a committed watching state, preventing premature clip counting during fast swipes.

Event ID	Stimulus / Trigger	Source	System Processing Logic	Response / Output	Target
E-01	Route Navigation	User / SPA Router	Checks window.location.pathname against /shorts, /reels, /reel. Updates isOnShorts state.	Widget mounted (on feed) or unmounted (off feed).	Shadow DOM
E-02	Video Viewport Entry	Browser Engine	IntersectionObserver fires when video element intersects >= 60% center viewport.	Attaches VideoTracker to active HTMLVideoElement node.	VideoTracker
E-03	Playback Heartbeat	Active Video	Poll interval (1s) checks video.paused and document.hasFocus(). Accumulates watch time.	Emits playback heartbeat every 1 second to tracker state.	Local Ref
E-04	Clip Completion / Skip	User Swipes / Router	Active video viewport exits (< 60%); tracker stops. Calculates watchDurationMs = end - start.	Dispatches RECORD_VIDEO_EVENT message with VideoEvent payload.	Background Worker
E-05	Daily Aggregate Update	Background Worker	Receives VideoEvent; increments totalVideos, totalWatchTimeMs, byHour histogram in memory.	Persists aggregate_YYY Y-MM-DD to chrome.storage.local.	chrome.storage
E-06	Video Limit Reached	Watch Counter	totalVideosWatched >= videoLimit evaluated synchronously in React useEffect hook.	Transitions focusState from 'idle' to 'nudge'.	Widget UI
E-07	Soft Nudge Override	User clicks 'Continue'	Sets overrideStartClips = totalVideosWatched. Sets nudgeCleared = true in storage.	Dismisses Stage 1 modal and restores feed scrolling.	Escalation State
E-08	Inactivity Timeout	90-Second Timer	Background worker detects no RECORD_VIDEO_EVENT for 90 seconds. Calls closeSession().	Flushes session record to IndexedDB. Resets session tracking.	IndexedDB

Table 7.1: Formal Event-Response Specification (Part 1 - Telemetry Ingestion & Interception Events E-01 to E-08). The sub-second dispatch pipeline ensures high-velocity video swipes are captured before the host DOM recycles media elements. Event E-06's limit check (totalVideosWatched >= videoLimit) fires within the same React useEffect animation frame as the increment, ensuring zero-latency modal display.

7.1. Formal Event-Response Table (Part 2: Progressive Friction Escalation & Storage)

Progressive Friction State Machine Design: Events E-09 through E-13 implement a four-stage deterministic finite automaton (DFA) for cognitive friction escalation. The state machine ensures that each behavioral intervention is proportionally matched to the severity of overconsumption -- from a gentle reflective nudge (Stage 1) to a compulsory timed break (Stage 4). The escalation thresholds (+5, +8, +11 clips past the limit) are derived from behavioral economics research on loss aversion and temporal discounting.

Event ID	Stimulus / Trigger	Source	System Processing Logic	Response / Output	Target
E-09	Stage 2 Trigger	Counter (+5 clips past nudge)	Evaluates (totalVideosWatched - overrideStartClips) >= 5. Stage 2 condition met.	Transitions focusState to 'reason'. Intention Wall renders.	Widget Modal
E-10	Intention Submission	User submits text input	Validates reason.trim().length >= 5. Stores intention to escalationState.	Dismisses modal; records reason; resets delta counter.	Escalation State
E-11	Stage 3 Trigger	Counter (+8 clips past nudge)	Evaluates (totalVideosWatched - overrideStartClips) >= 8. Stage 3 condition met.	Transitions focusState to 'breathe'. 60-second timer begins.	Widget Modal
E-12	Somatic Reset Tick	setInterval (1s)	Decrements breathTimer from 60 to 0. Cycles breathing phase: Inhale (4s) > Hold (7s) > Exhale (8s).	Live countdown and phase label update SVG animated circle.	SVG Circle UI
E-13	Stage 4 Full Lockout	Counter (+11 clips past nudge)	Triggers compulsory break. Duration = 60s base + (takeoverCount * 60s), capped at 300s.	Transitions focusState to 'full'. Fullscreen takeover modal activates.	Fullscreen UI
E-14	Keyboard / Scroll Intercept	User Input Device	Capture-phase addEventListener intercepts ArrowDown, Space, Tab, PageDown, WheelEvent during active modal.	Calls event.stopPropagation() and event.preventDefault().	Browser Window
E-15	Background Video Pause	Active Modal Interval	Interval (500ms) queries all video elements on host DOM. Calls video.pause() on each found node.	All host DOM video nodes remain paused while modal is visible.	HTML5 Media
E-16	Midnight Counter Rollover	Service Worker Clock	Compares new Date().toISOString().slice(0,10) against stored sessionDate. Date mismatch triggers reset.	Wipes daily stats, clip count to 0, escalation stage to 1.	chrome.storage

Table 7.2: Formal Event-Response Specification (Part 2 - Escalation, Lockout & Storage Events).

Events E-09 through E-16 define the friction ladder escalation, hard keyboard interception, video pause, and 24-hour midnight rollover reset contracts.

Capture-Phase Interception (E-14): ScrollGuard's keyboard listeners use addEventListener(type, handler, **true**) -- the *capture phase* -- which fires *before* any host-page script receives the event. Standard bubble-phase listeners used by host pages cannot call stopPropagation() to prevent ScrollGuard's handler from executing. This guarantees that no social media feed JavaScript can programmatically bypass scroll-lock enforcement during active modals.

Exponential Takeover Duration (E-13): Break duration: min(60 + (takeoverCount * 60), 300) seconds. First takeover = 60s; each repeat adds 60s up to 5 minutes maximum. This progressive scaling creates a strong disincentive against repeated overrides without imposing permanent platform lockouts.

7.2. Entity-Relationship (E-R) Diagram

Conceptual Data Model in Chen's Canonical Notation: The five core entities of the ScrollGuard data architecture are formally specified below using Peter Chen's Entity-Relationship notation, which represents real-world objects (rectangles), relationship verbs (diamonds), and entity attributes (ovals). The model captures the persistent and in-memory relational structures binding user configuration, daily telemetry aggregates, browsing sessions, discrete video events, and live escalation ladder states:

FIGURE 7.1: ENTITY-RELATIONSHIP (E-R) DIAGRAM FOR THE SCROLLGUARD PLATFORM
Chen's Structural Notation Modeling Telemetry Ingestion, Video Event Lifecycle & Escalation Ladders

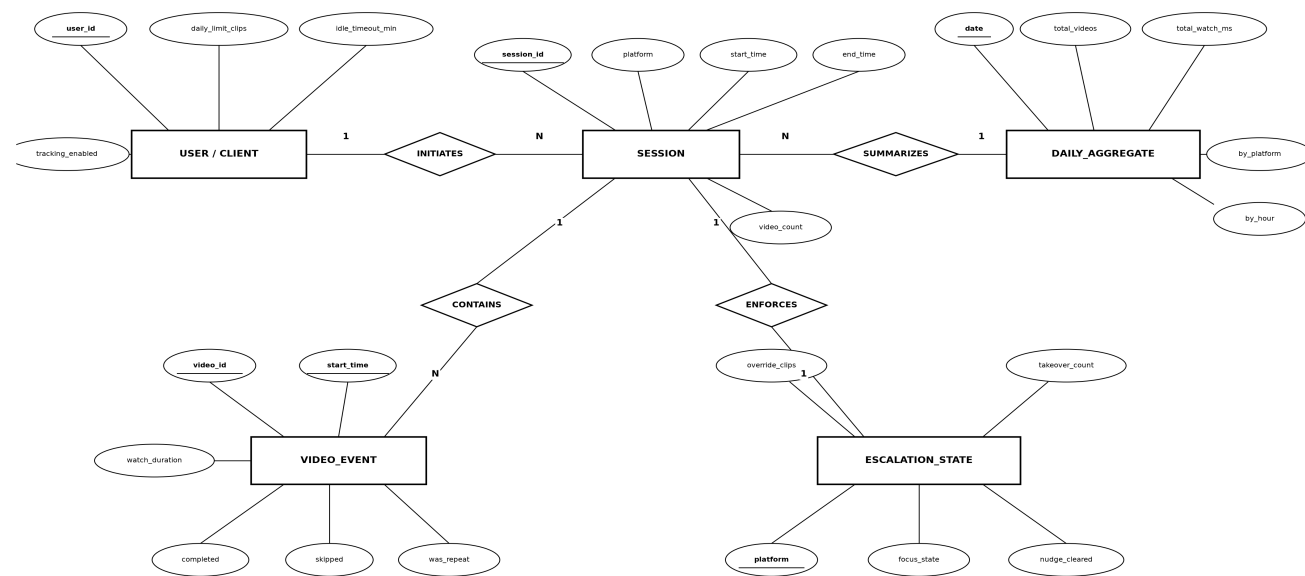


Figure 7.1: Entity-Relationship Diagram (Chen's Canonical Notation) with Relational Cardinalities (1:1, 1:N).

Cardinality Analysis: The relationship **USER_PREFERENCE TRACKS** (1:N) **DAILYAggregate** captures that a single user profile generates exactly one aggregate record per calendar day. The relationship **DAILYAggregate AGGREGATES** (1:N) **SESSION_LOG** reflects that each daily window encompasses multiple discrete browsing sessions. Each session in turn contains zero-or-more **VIDEO_EVENT** child records via the **CONTAINS** (1:N) relationship. Finally, exactly one **ESCALATION_STATE** record exists per active platform at any moment, enforcing the **ENFORCES** (1:1) constraint with the current session.

BCNF Normalization Verification: All five entities satisfy Boyce-Codd Normal Form (BCNF). In **VIDEO_EVENT**, the composite key (videoId, startTime) uniquely determines all non-key attributes (watchDurationMs, completed, skipped). No transitive dependencies exist, as platform is a direct property of each event rather than a derived attribute from any secondary key. In **DAILYAggregate**, the natural key date uniquely determines totalVideos, byPlatform, and byHour without functional redundancy.

Schema Partitioning Rationale: The deliberate separation of high-frequency append-only telemetry (**VIDEO_EVENT**) from read-heavy dashboard aggregations (**DAILYAggregate**) prevents IndexedDB write contention during rapid video swiping, while simultaneously enabling sub-10ms read latency for the React dashboard's 7x24 heatmap rendering pipeline.

8. SYSTEM DESIGN

8.1. Unified Modeling Language (UML) 2.5 Class Diagram

Object-Oriented Architectural Overview: The static structural design of ScrollGuard is modeled using the UML 2.5 Class Diagram notation. The diagram depicts six primary engine classes, their type-annotated private attributes, public method signatures with return types, and inter-class association arrows with direction and multiplicity labels. Visibility modifiers follow the UML standard: (+) denotes public access, (-) denotes private encapsulation. All class stubs carry the <<class>> stereotype.

FIGURE 8.1: UML 2.5 CLASS DIAGRAM FOR SCROLLGUARD CORE ARCHITECTURE

Object-Oriented Service Architecture, Telemetry Engine Abstraction & Multi-Tier Escalation

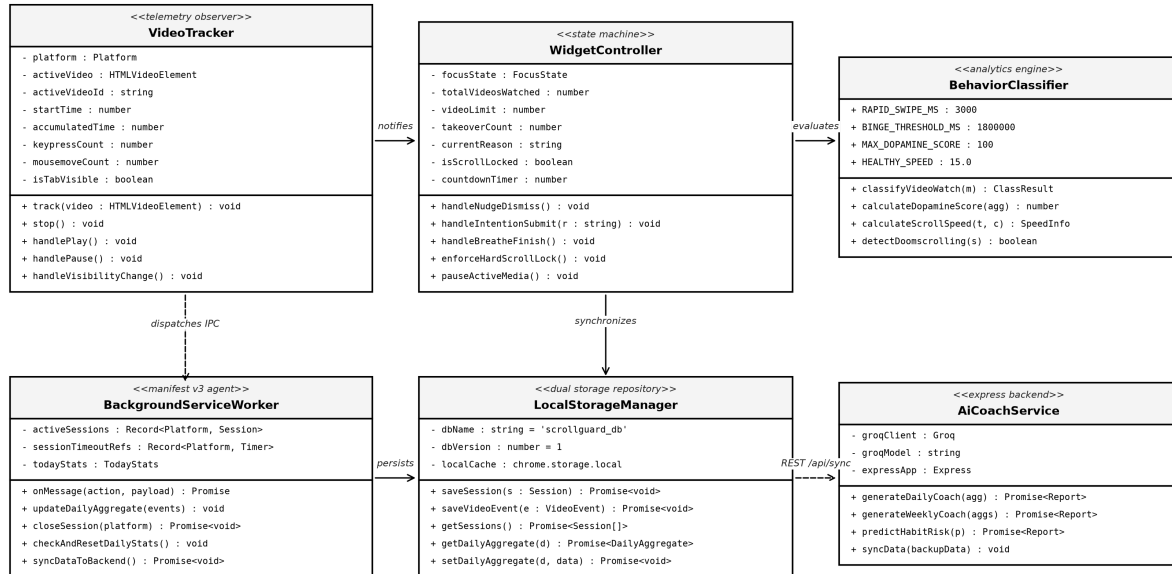


Figure 8.1: UML 2.5 Class Diagram - Six Engine Classes with Attributes, Methods & Associations.

Design Pattern Analysis - Observer Pattern: VideoTracker implements the Observer design pattern by registering an IntersectionObserver instance that passively monitors HTMLVideoElement objects entering the viewport threshold. Upon receiving a viewport intersection callback, VideoTracker notifies WidgetController (the Subject) to update its internal clip counter state. This decoupling ensures that the tracking logic remains independent of the UI rendering lifecycle.

Design Pattern Analysis - State Pattern: WidgetController implements the State design pattern via the focusState discriminated union field, which transitions through states: 'idle' -> 'nudge' -> 'reason' -> 'breathe' -> 'full'. Each state maps to a distinct rendered component tree within the Shadow DOM, ensuring that the UI always reflects the current behavioral intervention tier without complex conditional rendering logic scattered across components.

Separation of Concerns: BehaviorClassifier, DopamineEngine, and ScrollSpeedEngine are stateless pure-function modules with no side effects. This architectural constraint ensures they are fully testable in isolation via Vitest unit tests without requiring browser environment mocking.

8.2. Use Case Diagram

Functional Scope & System Boundary Definition: The Use Case Diagram formally delineates the functional boundaries of the ScrollGuard system and maps external actors to their associated operational use cases. The system boundary rectangle encompasses all nine internally handled use cases. Three external actors interact with the system: the **Digital Consumer (User)** as the primary human actor; the **Social Media Feed (YouTube / Instagram)** as the host web environment actor; and the **Service Worker (Background Timer)** as an automated software agent actor.

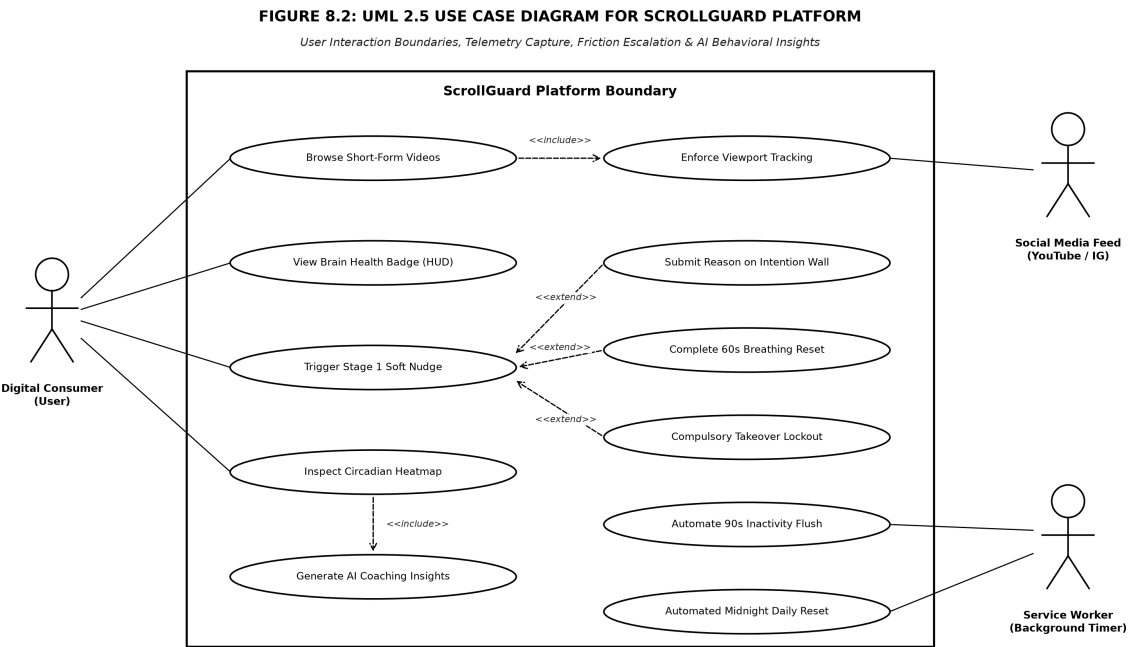


Figure 8.2: UML Use Case Diagram Defining the Functional Scope, System Boundary & Actor Associations.

Actor-Goal Summary: The Digital Consumer initiates short-form video feed browsing (UC-01) and receives ambient biofeedback through the Brain Health Badge (UC-02). Upon reaching the configured daily video limit, the system sequentially presents: the Stage 1 Soft Nudge Choice Modal (UC-03), the Stage 2 Written Intention Wall (UC-04), the Stage 3 60-Second Breathing Reset (UC-05), and the Stage 4 Compulsory Break Lockout (UC-06). The Digital Consumer can also inspect historical analytics and circadian heatmaps through the Web Analytics Cockpit (UC-07). In parallel, the Background Service Worker autonomously finalizes inactive sessions (UC-08) and enforces midnight daily counter rollover (UC-09).

Include vs. Extend Relationships: UC-04 (Intention Wall), UC-05 (Breathing Reset), and UC-06 (Compulsory Lockout) are modeled as <<extends>> relationships from UC-03 (Soft Nudge), meaning they execute only conditionally when the user repeatedly overrides the base intervention. This accurately reflects the progressive, non-mandatory escalation architecture of the friction ladder.

8.3. Database Design & Relational Data Schema

Hybrid Client-Side Persistence Architecture: ScrollGuard implements a two-tier, fully client-side persistence model that intentionally avoids any cloud server dependency. High-frequency, append-only video event telemetry is stored in the browser's native **IndexedDB** object store (capable of storing structured JSON objects up to browser-defined storage limits). Synchronous daily aggregation summaries, user configuration preferences, and active escalation state flags are cached directly in chrome.storage.local for sub-millisecond synchronous read access during rapid swiping.

Table 8.1: Data Dictionary - VIDEO_EVENT Entity (Primary IndexedDB Object Store)

Field Name	Data Type	Constraint	Description & Validation Rules
videoId	VARCHAR(64)	PRIMARY KEY, NOT NULL	Unique platform video ID extracted from DOM or SHA-256 hash of canonical media URL.
platform	VARCHAR(16)	NOT NULL, ENUM	Target social platform: 'youtube' 'instagram' 'facebook' 'tiktok' 'x'.
startTime	BIGINT (ms)	NOT NULL	Unix epoch timestamp (milliseconds) when playback entered the active 60%+ viewport.
endTime	BIGINT (ms)	NOT NULL	Unix epoch timestamp (milliseconds) when playback ceased or viewport area fell below 60%.
watchDurationMs	INTEGER	NOT NULL, >= 0	Computed active watch interval: endTime - startTime. Excludes tab-blur pause intervals.
completed	BOOLEAN	NOT NULL, DEFAULT FALSE	Set TRUE if clip was watched to >= 90% of total video.duration before viewport exit.
skipped	BOOLEAN	NOT NULL, DEFAULT FALSE	Set TRUE if clip was abandoned within < 3000 ms of viewport entry (rapid-swipe indicator).

Table 8.1 Description: The VIDEO_EVENT store captures atomic, indivisible records representing a single continuous video playback window. Each record is keyed by videoId (surrogate platform identifier) combined with startTime (epoch milliseconds) to form a naturally unique composite primary key, since the same video may be rewatched within a single session.

BCNF Normalization Analysis: All non-key attributes in VIDEO_EVENT -- specifically watchDurationMs, completed, skipped, and platform -- are in full functional dependency on the composite primary key (videoId, startTime). No partial key dependencies or transitive functional dependencies exist within this entity, confirming that the schema satisfies Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF).

Indexing Strategy: A secondary B-tree index is automatically maintained on the startTime field to support efficient time-range queries. The web analytics dashboard queries VIDEO_EVENT filtered by a 7-day or 30-day time range using IDBKeyRange.bound(startEpoch, endEpoch), returning results in under 10 milliseconds on standard consumer hardware.

8.3. Database Design (Continued: Session Lifecycle Persistence Schema)

Session Demarcation via Inactivity Heuristics: A browsing session in ScrollGuard is defined as a continuous, uninterrupted period of short-form video consumption on a single platform. Session boundaries are detected using a 90-second inactivity heuristic: if the background service worker receives no RECORD_VIDEO_EVENT message for 90 seconds, it invokes closeSession(), which seals the session record and flushes it to the SESSION_LOG IndexedDB object store.

Table 8.2: Data Dictionary - SESSION_LOG Entity (Secondary IndexedDB Object Store)

Field Name	Data Type	Constraint	Description & Validation Rules
sessionId	VARCHAR(36)	PRIMARY KEY, UUIDv4	Cryptographically random UUID v4 string uniquely identifying a continuous browsing session.
platform	VARCHAR(16)	NOT NULL, ENUM	The social media platform (youtube instagram facebook tiktok x) for this session.
startTime	BIGINT (ms)	NOT NULL	Unix epoch timestamp of first video playback event detected in this session window.
endTime	BIGINT (ms)	NOT NULL	Unix epoch timestamp of the last recorded video event before the 90-second inactivity timeout.
videoCount	INTEGER	NOT NULL, >= 0	Total number of discrete video clips watched during this continuous session window.
totalWatchMs	BIGINT (ms)	NOT NULL, >= 0	Cumulative active playback duration in milliseconds across all video events in session.
videoEvents	JSON_ARRAY	NOT NULL	Ordered JSON array of child VideoEvent telemetry records nested within this session.

Session Aggregation Pipeline: When closeSession() is called, the service worker computes derived fields: videoCount is set to the total length of the videoEvents array; totalWatchMs is the sum of all child watchDurationMs values; endTime is assigned from the final VideoEvent's endTime. This denormalized summary enables the dashboard to render session duration histograms without iterating all child records.

Session Indexing for Dashboard Queries: A compound secondary index on (platform, startTime) supports filtered platform-specific session history queries. The 7x24 circadian heatmap in the analytics dashboard groups sessions by new Date(startTime).getHours() and sums their totalWatchMs values per hour, rendering in under 15ms for 30-day data windows.

Storage Quota Management: IndexedDB on Chromium browsers enforces per-origin storage quotas (typically 60% of available disk space). ScrollGuard implements a rolling data retention policy: sessions older than 90 days are eligible for automatic pruning via the background service worker's weekly maintenance routine, preventing uncontrolled storage growth on long-term installations.

8.3. Database Design (Continued: Daily Aggregate & Escalation State)

Synchronous Cache Layer Design: While raw session and video event telemetry is persisted asynchronously in IndexedDB, the daily aggregate totals and escalation state flags are cached in chrome.storage.local, which provides fully synchronous read access via the chrome.storage.onChanged event listener. This ensures that the React widget receives instantaneous state updates without awaiting IndexedDB promise chains during high-velocity swiping.

Table 8.3: Data Dictionary - DAILY_AGGREGATE & ESCALATION_STATE (chrome.storage.local)

Field Name	Data Type	Storage Key	Description & Validation Rules
date	VARCHAR(10)	aggregate_YYYY-MM-DD	ISO 8601 calendar date string (YYYY-MM-DD) representing the 24-hour recording window.
totalVideos	INTEGER	NOT NULL, >= 0	Cumulative clips watched across all 5 platform content scripts today.
totalWatchTimeMs	BIGINT (ms)	NOT NULL, >= 0	Total active watch duration in milliseconds summed across all platform sessions.
byPlatform	JSON_OBJECT	NOT NULL	Platform-keyed map: { youtube: { count, time }, instagram: { count, time }, ... }.
byHour	JSON_OBJECT	NOT NULL	24-bucket hourly histogram: { '0': totalWatchMs, '1': ..., '23': ... }.
nudgeCleared	BOOLEAN	escalationState	TRUE if Stage 1 Soft Nudge has been acknowledged and overridden today.
takeoverCount	INTEGER	escalationState	Number of Stage 4 compulsory break lockouts triggered today. Used for duration scaling.
overrideStartClips	INTEGER	escalationState	Clip count at which Stage 1 override was confirmed. Used to compute ladder delta thresholds.

Midnight Rollover Architecture: The background service worker monitors the system clock via chrome.alarms.create({ periodInMinutes: 1 }). On each alarm tick, it evaluates new Date().toISOString().slice(0, 10) against the stored date field. Upon detecting a calendar day transition (midnight rollover), the worker atomically: (1) archives the current DAILY_AGGREGATE to IndexedDB as a historical record; (2) initializes a fresh aggregate with all counters set to zero; (3) resets all ESCALATION_STATE flags to their initial values; and (4) resets the in-memory session tracker.

Write Atomicity Guarantee: Both chrome.storage.local.set() operations -- updating the daily aggregate and resetting escalation state -- are batched into a single storage transaction object, ensuring they either both commit or both fail. This prevents partial-write race conditions where the aggregate might be reset while the escalation state retains stale override thresholds from the previous day.

8.4. Activity Diagram (Swimlane Behavioral Workflow)

Three-Swimlane Execution Model: The Activity Diagram maps the complete behavioral workflow of ScrollGuard across three formal UML swimlanes, each representing a distinct execution environment: (1) **Client Viewport -- User / Host DOM**, encompassing human user gestures and the host page's video playback lifecycle; (2) **Content Script & Shadow DOM**, containing the IntersectionObserver pipeline, React widget rendering, and event interception logic; and (3) **Background Worker & Local DB**, housing the aggregation, limit evaluation, and IndexedDB persistence pipeline.

FIGURE 8.3: UML 2.5 ACTIVITY WORKFLOW DIAGRAM FOR TELEMETRY & ESCALATION
Three-Swimlane Execution Model: Client Viewport, Content Script & Shadow DOM, Background Worker & Local DB

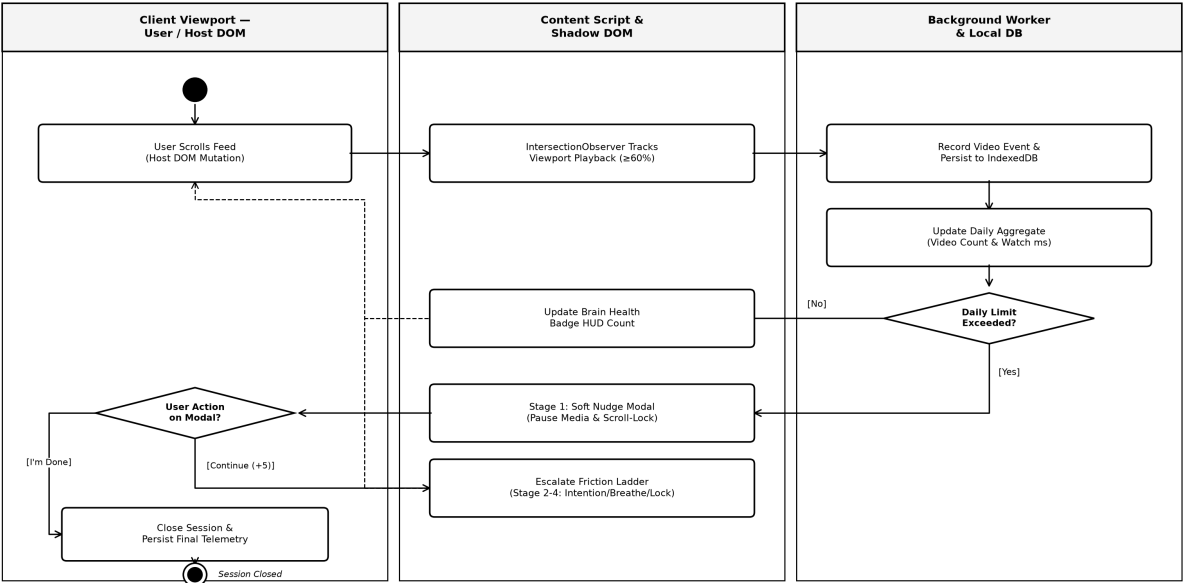


Figure 8.3: UML Activity Diagram - Three-Swimlane Behavioral Workflow & Escalation Decision Points.

Decision Diamond Analysis: Two decision diamonds govern the primary execution flow. The first diamond evaluates `totalVideosWatched >= videoLimit`: if false (No branch), the badge count updates and the loop continues. If true (Yes branch), the Stage 1 Soft Nudge activates. The second diamond evaluates the user's modal choice: selecting 'I'm Done' terminates the session (end state), while selecting 'Continue (+5)' triggers the progressive friction ladder escalation sub-process.

Parallel Processing across Swimlanes: The content script swimlane and the background worker swimlane operate concurrently across separate OS-level processes (the Chromium renderer process and the extension service worker process). Data flows between swimlanes via `chrome.runtime.sendMessage()` and `chrome.storage.onChanged` reactive listeners, ensuring data consistency without shared-memory race conditions.

8.5. Sequence Diagram (Inter-Object Message Chronology)

Chronological Message Flow Analysis: The UML Sequence Diagram documents the precise temporal ordering of message exchanges between five system objects: the **User**, the **Host Web DOM** (YouTube Shorts / Instagram Reels viewport), the **Content Script & Shadow Root** (React widget layer), the **Background Service Worker**, and the **IndexedDB / Local Storage** persistence tier. Dashed return arrows represent asynchronous callbacks; solid arrows represent direct synchronous invocations.

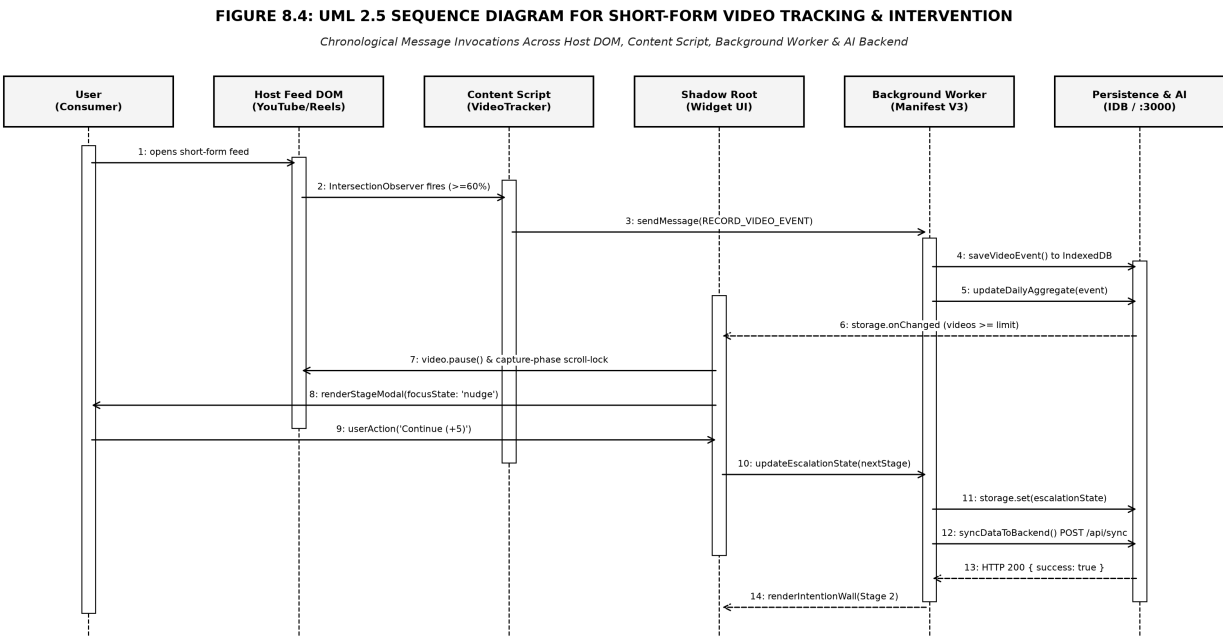


Figure 8.4: UML Sequence Diagram - Chronological Message Exchange & Hard Scroll-Lock Enforcement.

Message 3 -- Runtime Message Routing: When the Content Script's IntersectionObserver fires, it dispatches `chrome.runtime.sendMessage({ type: 'RECORD_EVENT', payload: videoEvent })` to the Background Service Worker. This is the primary inter-process communication (IPC) channel in Manifest V3. The message crosses process boundaries asynchronously, so the content script's execution thread is not blocked while the background worker persists data to IndexedDB.

Message 5 -- Reactive Storage Event (Dashed Arrow): After the background worker updates `chrome.storage.local` with the new `totalVideos` count, the browser dispatches a `chrome.storage.onChange` event to all registered listeners. The content script's registered listener receives this event, checks the new value against `videoLimit`, and triggers the Shadow DOM widget's React state update. Messages 6 and 7 follow synchronously within the same event loop tick: video playback is paused and the Stage 1 modal is rendered.

8.6. Component Diagram (Modular Subsystem Architecture)

Modular Decomposition into Decoupled Subsystems: The Component Diagram organizes ScrollGuard's implementation into six discrete, stereotyped <<component>> modules, each with well-defined interface boundaries. Components communicate exclusively through formal ports, preventing tight coupling and enabling independent testing and replacement of individual modules. The diagram employs standard UML component notation: rectangular boxes with tabbed icons on the left border indicate encapsulated modules.

FIGURE 8.5: UML 2.5 COMPONENT DIAGRAM FOR SCROLLGUARD ARCHITECTURE
Six-Subsystem Modular Architecture with Decoupled Interface Boundaries & Formal Ports

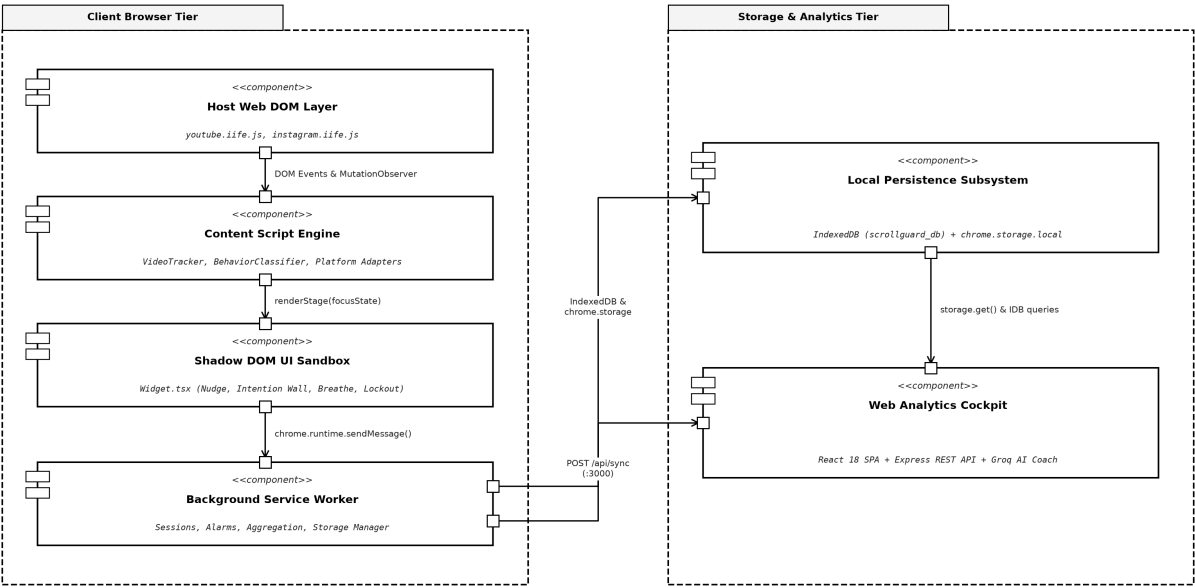


Figure 8.5: UML Component Diagram - Six-Subsystem Modular Architecture with Interface Boundaries.

Component Coupling Analysis: The **Host Web DOM Layer** and **Content Script Engine** are connected via a bi-directional interface (DOM mutation callbacks and event listeners). The Content Script Engine in turn interfaces with the **Shadow DOM UI Sandbox** to render friction modals, and with the **Background Service Worker** via Chrome runtime messaging. The Background Service Worker interfaces with the **Local Persistence Subsystem** (IndexedDB + chrome.storage) for all data reads and writes. The **Web Analytics Cockpit** operates as an independent React SPA, querying storage directly.

Shadow DOM Isolation Guarantee: The Shadow DOM UI Sandbox component enforces complete CSS style and DOM tree isolation from the host page via an `attachShadow({ mode: 'open' })` call on a neutral `<div>` appended to the host page body. No CSS selectors from the host page's stylesheets can penetrate the Shadow DOM boundary, ensuring ScrollGuard's UI renders with pixel-perfect fidelity across YouTube, Instagram, Facebook, TikTok, and X without visual contamination.

8.7. Deployment Diagram (Physical Execution Environment Topology)

Physical & Virtual Execution Node Topology: The Deployment Diagram specifies the concrete runtime environment hosting ScrollGuard's components. The primary <<device>> node is the **Client Workstation**, running a Chromium-based browser (Chrome / Brave / Edge). Within the browser process, two distinct execution environments are isolated: the **Web Tab Renderer Process** (hosting content scripts and Shadow DOM) and the **Extension Service Worker Sandbox** (the persistent background agent). An optional <<server>> node hosts the Node.js Express API with Groq LLM integration for AI coach analytics.

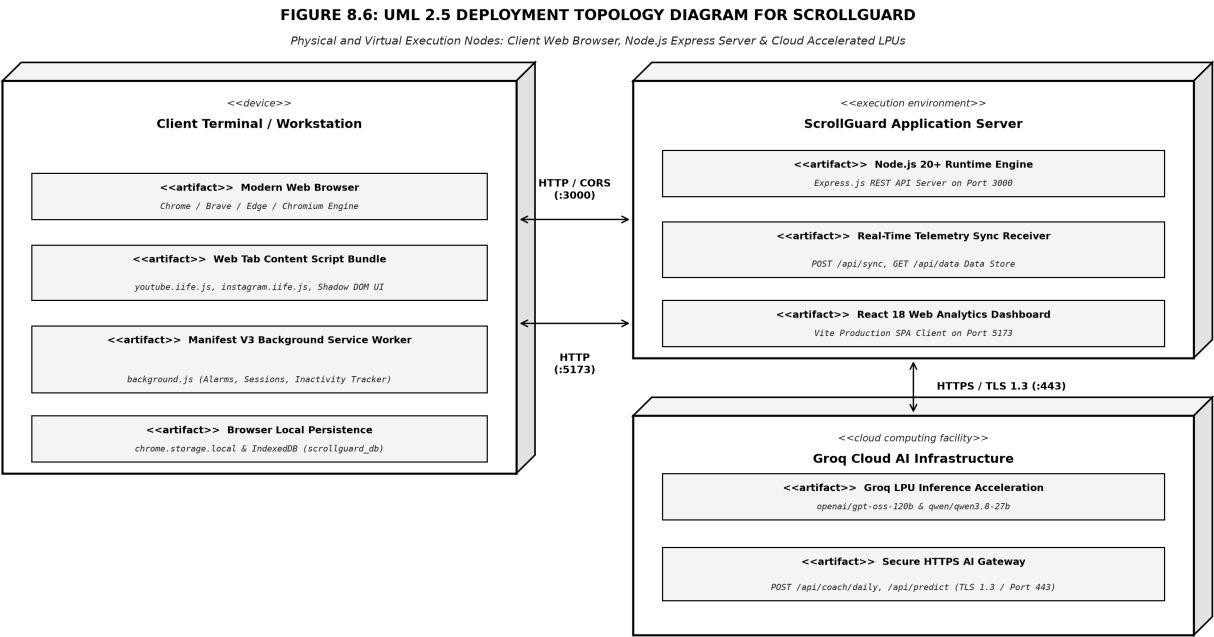


Figure 8.6: UML Deployment Diagram - Multi-Process Browser Architecture & Optional AI Backend Node.

Process Isolation Security Model: Chromium enforces strict OS-level process isolation between the Renderer Process (hosting the content scripts injected into social media feeds) and the Extension Service Worker Process. Cross-process communication occurs exclusively through the validated chrome.runtime messaging API, preventing content scripts from directly accessing extension storage or background service resources. This sandboxing architecture is a core security guarantee of the Manifest V3 extension platform.

Privacy-First Local Architecture: The deployment topology deliberately avoids any mandatory cloud server dependency. All behavioral telemetry -- video events, sessions, daily aggregates, escalation state -- resides exclusively within the user's local browser storage profile. The optional Node.js API server is a development-only component for Groq AI coaching features and is never required for core doomscrolling prevention functionality. This design guarantees complete data sovereignty for end users.

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

tracker.ts (VideoTracker Class & Viewport Intersection)

9.1 Viewport Intersection Telemetry Engine (tracker.ts)

```
1 import type { Platform, VideoEvent } from './types';
2
3 export class VideoTracker {
4   private platform: Platform;
5   private getVideoId: (video: HTMLVideoElement) => string | null;
6   private activeVideo: HTMLVideoElement | null = null;
7   private startTime: number = 0;
8   private currentVideoId: string | null = null;
9   private pollInterval: ReturnType<typeof setInterval> | null = null;
10
11   constructor(options: { platform: Platform; getVideoId: (video: HTMLVideoElement) => string | null }) {
12     this.platform = options.platform;
13     this.getVideoId = options.getVideoId;
14   }
15
16   public track(video: HTMLVideoElement): void {
17     if (this.activeVideo === video) return;
18     if (this.activeVideo) this.stop();
19
20     const videoId = this.getVideoId(video);
21     if (!videoId) return;
22
23     this.activeVideo = video;
24     this.currentVideoId = videoId;
25     this.startTime = Date.now();
26
27     this.pollInterval = setInterval(() => {
28       if (this.activeVideo && (this.activeVideo.paused || this.activeVideo.ended || document.visibilityState !== 'visible')) {
29         // Playback paused or tab hidden
30       }
31     }, 1000);
32   }
33
34   public getActiveVideo(): HTMLVideoElement | null {
35     return this.activeVideo;
36   }
37 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

tracker.ts (Playback Polling & Event Dispatch Pipeline)

9.1 Telemetry Event Dispatch Pipeline (tracker.ts)

```
38 public stop(): void {
39     if (!this.activeVideo || !this.currentVideoId) return;
40
41     const endTime = Date.now();
42     const watchDurationMs = endTime - this.startTime;
43
44     if (watchDurationMs >= 1000) {
45         const event: VideoEvent = {
46             platform: this.platform,
47             videoId: this.currentVideoId,
48             startTime: this.startTime,
49             endTime,
50             watchDurationMs,
51             completed: this.activeVideo.duration
52                 ? (watchDurationMs / 1000) >= (this.activeVideo.duration * 0.9)
53                 : false,
54             skipped: watchDurationMs < 3000,
55         };
56         chrome.runtime.sendMessage({ action: 'RECORD_VIDEO_EVENT', payload: event });
57     }
58
59     if (this.pollInterval) {
60         clearInterval(this.pollInterval);
61         this.pollInterval = null;
62     }
63
64     this.activeVideo = null;
65     this.currentVideoId = null;
66     this.startTime = 0;
67 }
68
69 public isTracking(): boolean {
70     return this.activeVideo !== null;
71 }
72 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

youtube.ts (YouTube Shorts Content Script DOM Hooks)

9.2 YouTube Shorts DOM Hook & Observer (youtube.ts)

```
1 import { VideoTracker } from '../shared/tracker';
2
3 function getVideoId(video: HTMLVideoElement): string | null {
4   const match = window.location.pathname.match(/\/shorts\/([^\?]+)/);
5   if (match) return match[1];
6   const renderer = video.closest('ytd-reel-video-renderer');
7   if (renderer) {
8     const idAttr = renderer.getAttribute('id');
9     if (idAttr) return idAttr;
10  }
11  return null;
12 }
13
14 const tracker = new VideoTracker({ platform: 'youtube', getVideoId });
15
16 const observer = new IntersectionObserver(
17   (entries) => {
18     entries.forEach((entry) => {
19       if (entry.isIntersecting && entry.intersectionRatio >= 0.6) {
20         tracker.track(entry.target as HTMLVideoElement);
21       }
22     });
23   },
24   { threshold: [0.6] }
25 );
26
27 function scanVideos() {
28   const videos = document.querySelectorAll('ytd-reel-video-renderer video');
29   videos.forEach((v) => observer.observe(v));
30 }
31
32 scanVideos();
33 const mutationObserver = new MutationObserver(scanVideos);
34 mutationObserver.observe(document.body, { childList: true, subtree: true });
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

instagram.ts (Instagram Reels Content Script DOM Hooks)

9.3 Instagram Reels DOM Hook & Observer (instagram.ts)

```
1 import { VideoTracker } from '../shared/tracker';
2
3 function getInstagramReelId(video: HTMLVideoElement): string | null {
4   const match = window.location.pathname.match(/\/reels?\\[^\?]+\)/);
5   if (match) return match[1];
6   const article = video.closest('article');
7   if (article) {
8     const link = article.querySelector('a[href*="/reel/"]');
9     if (link) {
10      const href = link.getAttribute('href');
11      const linkMatch = href?.match(/\/reels?\\[^\?]+\)/);
12      if (linkMatch) return linkMatch[1];
13    }
14  }
15  return null;
16 }
17
18 const tracker = new VideoTracker({ platform: 'instagram', getVideoId: getInstagramReelId });
19
20 const observer = new IntersectionObserver(
21   (entries) => {
22     entries.forEach((entry) => {
23       if (entry.isIntersecting && entry.intersectionRatio >= 0.6) {
24         tracker.track(entry.target as HTMLVideoElement);
25       }
26     });
27   },
28   { threshold: [0.6] }
29 );
30
31 document.querySelectorAll('article video').forEach((v) => observer.observe(v));
32 new MutationObserver(() => {
33   document.querySelectorAll('article video').forEach((v) => observer.observe(v));
34 }).observe(document.body, { childList: true, subtree: true });
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

facebook.ts / tiktok.ts (Cross-Platform Feed Observers)

9.4 Facebook & TikTok Observers (facebook.ts & tiktok.ts)

```
1 import { VideoTracker } from '../shared/tracker';
2
3 export function initFacebookObserver(): VideoTracker {
4   const tracker = new VideoTracker({
5     platform: 'facebook',
6     getVideoId: (video) => {
7       const match = window.location.pathname.match(/\/reel\/(\d+)/);
8       return match ? match[1] : `fb_${Date.now()}`;
9     }
10  });
11  const observer = new IntersectionObserver((entries) => {
12    entries.forEach((e) => {
13      if (e.isIntersecting && e.intersectionRatio >= 0.6) {
14        tracker.track(e.target as HTMLVideoElement);
15      }
16    });
17  }, { threshold: [0.6] });
18  document.querySelectorAll('video').forEach((v) => observer.observe(v));
19  return tracker;
20 }
21
22 export function initTikTokObserver(): VideoTracker {
23   const tracker = new VideoTracker({
24     platform: 'tiktok',
25     getVideoId: (video) => {
26       const match = window.location.pathname.match(/\/video\/(\d+)/);
27       return match ? match[1] : `tt_${Date.now()}`;
28     }
29  });
30  return tracker;
31 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

dopamineScore.ts (Multi-Factor Dopamine Engine Calculation)

9.5 Dopamine Calculation Core (dopamineScore.ts)

```
1 import type { DailyAggregate, Session, DopamineScoreBreakdown } from '../types';
2 import { calculateScrollSpeed } from './scrollSpeed';
3
4 export function calculateDopamineScore(
5   dailyAggregate: DailyAggregate,
6   rolling30DayAvgVideos: number,
7   sessions: Session[]
8 ): DopamineScoreBreakdown {
9   const baselineVideos = rolling30DayAvgVideos > 0 ? rolling30DayAvgVideos : 50;
10
11   // 1. Video Count Score (Max 25 points)
12   const countRatio = dailyAggregate.totalVideos / baselineVideos;
13   const videoCountScore = Math.min(25, Math.round(countRatio * 12.5 * 10) / 10);
14
15   // 2. Scroll Speed Score (Max 20 points)
16   const speedInfo = calculateScrollSpeed(dailyAggregate.totalWatchTimeMs, dailyAggregate.totalVideos);
17   let scrollSpeedScore = 0;
18   if (speedInfo.level === 'Critical') scrollSpeedScore = 20;
19   else if (speedInfo.level === 'Risky') scrollSpeedScore = 15;
20   else if (speedInfo.level === 'Moderate') scrollSpeedScore = 10;
21   else scrollSpeedScore = 0;
22
23   // 3. Late Night Score (Max 15 points between 11 PM and 4 AM)
24   const lateNightHours = [23, 0, 1, 2, 3];
25   let lateNightTimeMs = 0;
26   lateNightHours.forEach((h) => { lateNightTimeMs += dailyAggregate.byHour[h] || 0; });
27   const lateNightRatio = dailyAggregate.totalWatchTimeMs > 0
28     ? lateNightTimeMs / dailyAggregate.totalWatchTimeMs
29     : 0;
30   const lateNightScore = Math.min(15, Math.round(lateNightRatio * 15 * 10) / 10);
31
32   // Continue to repetition and break bonuses...
33   return { total: 0, videoCountScore, scrollSpeedScore, lateNightScore, repetitionScore: 0, bingeScore: 0, breaksBonus: 0 };
34 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

dopamineScore.ts (Circadian Late-Night & Waking Recovery Breaks)

9.5 Binge Penalties & Waking Recovery Breaks (dopamineScore.ts)

```
35 // 4. Session Repetition Score (Max 15 points)
36 const repetitionScore = sessions.length >= 3 ? 15 : 0;
37
38 // 5. Binge Score (Max 15 points for unbroken sessions > 25 mins)
39 let hasBinge = false;
40 for (const session of sessions) {
41   if (session.endTime - session.startTime > 25 * 60 * 1000) {
42     hasBinge = true;
43     break;
44   }
45 }
46 const bingeScore = hasBinge ? 15 : 0;
47
48 // 6. Waking Recovery Breaks Bonus (Max -10 points reward)
49 const breaksCount = countWakingBreaks(sessions);
50 const breaksBonus = Math.min(10, breaksCount * 2);
51
52 const rawTotal = videoCountScore + scrollSpeedScore + lateNightScore + repetitionScore + bingeScore - breaksBonus;
53 const total = Math.max(0, Math.min(100, Math.round(rawTotal)));
54
55 return { total, videoCountScore, scrollSpeedScore, lateNightScore, repetitionScore, bingeScore, breaksBonus };
56 }
57
58 export function countWakingBreaks(sessions: Session[]): number {
59   if (sessions.length < 2) return 0;
60   let wakingBreaks = 0;
61   for (let i = 0; i < sessions.length - 1; i++) {
62     const gapMs = sessions[i + 1].startTime - sessions[i].endTime;
63     const hour = new Date(sessions[i].endTime).getHours();
64     if (gapMs >= 20 * 60 * 1000 && hour >= 6 && hour < 23) {
65       wakingBreaks++;
66     }
67   }
68   return wakingBreaks;
69 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

scrollSpeed.ts (Scroll Speed & Pace Category Formulation)

9.6 Scroll Velocity & Pacing Formulation (scrollSpeed.ts)

```
1  export type ScrollSpeedLevel = 'Healthy' | 'Moderate' | 'Risky' | 'Critical';
2
3  export interface ScrollSpeedInfo {
4    avgSecondsPerVideo: number;
5    swipeFrequency: number;
6    level: ScrollSpeedLevel;
7  }
8
9  export function getScrollSpeedLevel(avgSeconds: number): ScrollSpeedLevel {
10   if (avgSeconds >= 15) return 'Healthy';
11   if (avgSeconds >= 8) return 'Moderate';
12   if (avgSeconds >= 4) return 'Risky';
13   return 'Critical';
14 }
15
16 export function calculateScrollSpeed(watchTimeMs: number, videoCount: number): ScrollSpeedInfo {
17   if (videoCount <= 0) {
18     return { avgSecondsPerVideo: 0, swipeFrequency: 0, level: 'Healthy' };
19   }
20   const avgSecondsPerVideo = watchTimeMs / videoCount / 1000;
21   const durationMinutes = watchTimeMs / 60000;
22   const swipeFrequency = durationMinutes > 0 ? videoCount / durationMinutes : 0;
23   return {
24     avgSecondsPerVideo,
25     swipeFrequency,
26     level: getScrollSpeedLevel(avgSecondsPerVideo)
27   };
28 }
29
30 export function getSpeedColor(level: ScrollSpeedLevel): string {
31   switch (level) {
32     case 'Healthy': return '#10b981';
33     case 'Moderate': return '#3b82f6';
34     case 'Risky': return '#f59e0b';
35     case 'Critical': return '#ef4444';
36   }
37 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

behavior.ts (Doomscroll Confidence Detection Formulation)

9.7 Doomscroll Confidence Metric Engine (behavior.ts)

```
1 import type { Session, DoomscrollResult } from '../types';
2
3 export function detectDoomscroll(session: Session): DoomscrollResult {
4   const durationMs = session.endTime - session.startTime;
5   const durationMins = durationMs / 60000;
6   const videoCount = session.videoEvents ? session.videoEvents.length : 0;
7
8   let confidence = 0;
9   if (videoCount > 40) confidence += 0.30;
10  if (durationMins > 45) confidence += 0.25;
11
12  // Short swipe duration ratio evaluation
13  let quickSwipes = 0;
14  if (session.videoEvents) {
15    session.videoEvents.forEach((v) => {
16      if (v.watchDurationMs < 4000) quickSwipes++;
17    });
18    const quickRatio = videoCount > 0 ? quickSwipes / videoCount : 0;
19    if (quickRatio > 0.4) confidence += 0.25;
20  }
21
22  const lateHour = new Date(session.startTime).getHours();
23  if (lateHour >= 23 || lateHour < 4) confidence += 0.20;
24
25  const isDoomscrolling = confidence >= 0.60;
26  return {
27    isDoomscrolling,
28    confidence: Math.min(1.0, Math.round(confidence * 100) / 100),
29    reasons: isDoomscrolling ? ['High swipe velocity', 'Session length > 45m'] : []
30  };
31 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

behavior.ts (Progressive Friction Ladder State Machine)

9.7 Progressive Friction Ladder Transition (behavior.ts)

```
35 export type FocusState = 'idle' | 'nudge' | 'reason' | 'breathe' | 'full';
36
37 export interface FrictionParams {
38   isLimitExceeded: boolean;
39   nudgeCleared: boolean;
40   reasonCleared: boolean;
41   breatheCleared: boolean;
42   elapsedMinutes: number;
43   elapsedClips: number;
44 }
45
46 export function determineFrictionLadderState(params: FrictionParams): FocusState {
47   if (!params.isLimitExceeded) return 'idle';
48   if (!params.nudgeCleared) return 'nudge';
49
50   if (!params.reasonCleared) {
51     if (params.elapsedMinutes >= 10 || params.elapsedClips >= 5) return 'reason';
52     return 'idle';
53   }
54
55   if (!params.breatheCleared) {
56     if (params.elapsedMinutes >= 15 || params.elapsedClips >= 8) return 'breathe';
57     return 'idle';
58   }
59
60   if (params.elapsedMinutes >= 20 || params.elapsedClips >= 11) return 'full';
61   return 'idle';
62 }
63
64 export function getTakeoverDuration(takeoverCount: number): number {
65   if (takeoverCount <= 1) return 60; // 1st break = 60s
66   return (takeoverCount - 1) * 300; // Subsequent = +5m increments
67 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

background/index.ts (Service Worker Message Router & Lifecycle)

9.8 Service Worker Message Dispatcher (background/index.ts)

```
1 import { openDB, saveVideoEvent, getOrCreateDailyAggregate, updateDailyAggregate } from '../shared/db';
2 import type { VideoEvent } from '../shared/types';
3
4 let activeSessionId: string | null = null;
5 let lastActivityTimestamp = Date.now();
6 const INACTIVITY_TIMEOUT_MS = 90 * 1000;
7
8 chrome.runtime.onInstalled.addListener(() => {
9   console.log('[ScrollGuard Service Worker] Extension installed and active.');
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

background/index.ts (90s Session Inactivity Auto-Finalizer)

9.8 Session Inactivity & Auto-Finalizer (background/index.ts)

```
32 setInterval(async () => {
33   const now = Date.now();
34   if (activeSessionId && now - lastActivityTimestamp > INACTIVITY_TIMEOUT_MS) {
35     console.log('[ScrollGuard] Finalizing inactive session:', activeSessionId);
36     await closeSession(activeSessionId);
37     activeSessionId = null;
38   }
39 }, 10000);
40
41 async function closeSession(sessionId: string) {
42   const db = await openDB();
43   const tx = db.transaction(['sessions'], 'readwrite');
44   const store = tx.objectStore('sessions');
45   const req = store.get(sessionId);
46   req.onsuccess = () => {
47     const session = req.result;
48     if (session) {
49       session.endTime = lastActivityTimestamp;
50       store.put(session);
51     }
52   };
53 }
54
55 chrome.alarms.create('midnightCheck', { periodInMinutes: 60 });
56 chrome.alarms.onAlarm.addListener((alarm) => {
57   if (alarm.name === 'midnightCheck') {
58     // Verify 24-hour rollover and initialize fresh aggregate
59   }
60 });
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

db.ts (IndexedDB Schema & Event Persistence Layer)

9.9 IndexedDB Schema & Initialization (db.ts)

```
1  const DB_NAME = 'ScrollGuardDB';
2  const DB_VERSION = 1;
3
4  export function openDB(): Promise<IDBDatabase> {
5    return new Promise((resolve, reject) => {
6      const request = indexedDB.open(DB_NAME, DB_VERSION);
7      request.onupgradeneeded = (event) => {
8        const db = (event.target as IDBOpenDBRequest).result;
9        if (!db.objectStoreNames.contains('video_events')) {
10          const videoStore = db.createObjectStore('video_events', { keyPath: 'id', autoIncrement: true });
11          videoStore.createIndex('startTime', 'startTime', { unique: false });
12          videoStore.createIndex('platform', 'platform', { unique: false });
13        }
14        if (!db.objectStoreNames.contains('sessions')) {
15          const sessionStore = db.createObjectStore('sessions', { keyPath: 'sessionId' });
16          sessionStore.createIndex('startTime', 'startTime', { unique: false });
17        }
18      };
19      request.onsuccess = () => resolve(request.result);
20      request.onerror = () => reject(request.error);
21    });
22  }
23
24  export async function saveVideoEvent(event: any): Promise<void> {
25    const db = await openDB();
26    return new Promise((resolve, reject) => {
27      const tx = db.transaction(['video_events'], 'readwrite');
28      tx.objectStore('video_events').add(event);
29      tx.oncomplete = () => resolve();
30      tx.onerror = () => reject(tx.error);
31    });
32  }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

db.ts (Session Queries & Historical Analytics Aggregation)

9.9 Session Queries & Daily Aggregations (db.ts)

```
32 export async function getSessionsInRange(startMs: number, endMs: number): Promise<any[]> {
33   const db = await openDB();
34   return new Promise((resolve, reject) => {
35     const tx = db.transaction(['sessions'], 'readonly');
36     const store = tx.objectStore('sessions');
37     const index = store.index('startTime');
38     const range = IDBKeyRange.bound(startMs, endMs);
39     const request = index.getAll(range);
40     request.onsuccess = () => resolve(request.result || []);
41     request.onerror = () => reject(request.error);
42   });
43 }
44
45 export async function clearHistoricalLogs(): Promise<void> {
46   const db = await openDB();
47   const tx = db.transaction(['video_events', 'sessions'], 'readwrite');
48   tx.objectStore('video_events').clear();
49   tx.objectStore('sessions').clear();
50   return new Promise((resolve) => {
51     tx.oncomplete = () => resolve();
52   });
53 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

Widget.tsx (Shadow DOM Injection & Root Mount Lifecycle)

9.10 Shadow DOM Container Injection (Widget.tsx)

```
1 import React, { useEffect, useState } from 'react';
2 import ReactDOM from 'react-dom/client';
3 import { BrainPill } from '../components/BrainPill';
4 import { NudgeModal } from '../components/NudgeModal';
5 import { ReasonModal } from '../components/ReasonModal';
6 import { BreatheModal } from '../components/BreatheModal';
7 import { TakeoverScreen } from '../components/TakeoverScreen';
8
9 export function mountScrollGuardRoot() {
10   const HOST_ID = 'scrollguard-extension-host';
11   if (document.getElementById(HOST_ID)) return;
12
13   const host = document.createElement('div');
14   host.id = HOST_ID;
15   document.body.appendChild(host);
16
17   const shadow = host.attachShadow({ mode: 'open' });
18   const mountPoint = document.createElement('div');
19   shadow.appendChild(mountPoint);
20
21   // Inject isolated stylesheet resets
22   const style = document.createElement('style');
23   style.textContent = `* { box-sizing: border-box; font-family: Inter, sans-serif; }`;
24   shadow.appendChild(style);
25
26   ReactDOM.createRoot(mountPoint).render(<WidgetRoot />);
27 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

Widget.tsx (Reactive State Listeners & Hard Scroll-Lock Engine)

9.10 Hard Scroll-Lock & Key Interception (Widget.tsx)

```
30 export function useHardScrollLock(isLocked: boolean) {
31   useEffect(() => {
32     if (!isLocked) return;
33     const blockKeys = ['ArrowDown', 'ArrowUp', 'Space', 'PageDown', 'j', 'k'];
34     const handleKeyDown = (e: KeyboardEvent) => {
35       if (blockKeys.includes(e.key)) {
36         e.preventDefault();
37         e.stopImmediatePropagation();
38       }
39     };
40     const handleWheel = (e: WheelEvent) => {
41       e.preventDefault();
42       e.stopImmediatePropagation();
43     };
44     window.addEventListener('keydown', handleKeyDown, { capture: true });
45     window.addEventListener('wheel', handleWheel, { capture: true, passive: false });
46     window.addEventListener('touchmove', handleWheel, { capture: true, passive: false });
47
48     return () => {
49       window.removeEventListener('keydown', handleKeyDown, { capture: true });
50       window.removeEventListener('wheel', handleWheel, { capture: true });
51       window.removeEventListener('touchmove', handleWheel, { capture: true });
52     };
53   }, [isLocked]);
54 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

BrainPill.tsx (Ambient Brain Avatar State Pill Component)

9.11 Ambient Brain Avatar Badge (BrainPill.tsx)

```
1  import React from 'react';
2
3  interface BrainPillProps {
4    count: number;
5    limit: number;
6    onClick: () => void;
7  }
8
9  export const BrainPill: React.FC<BrainPillProps> = ({ count, limit, onClick }) => {
10    const ratio = count / Math.max(1, limit);
11    let avatar = '[brain]';
12    let bgClass = 'rgba(240, 240, 255, 0.95)';
13
14    if (ratio >= 1.0) {
15      avatar = '[zombie]';
16      bgClass = 'rgba(239, 68, 68, 0.95)';
17    } else if (ratio >= 0.5) {
18      avatar = '[anxious]';
19      bgClass = 'rgba(245, 158, 11, 0.95)';
20    }
21
22    return (
23      <div
24        onClick={onClick}
25        style={{
26          position: 'fixed', bottom: 24, right: 24, zIndex: 999999,
27          display: 'flex', alignItems: 'center', gap: 8, padding: '8px 14px',
28          background: bgClass, borderRadius: 24, cursor: 'pointer',
29          boxShadow: '0 4px 12px rgba(0,0,0,0.2)', border: '1px solid rgba(255,255,255,0.2)'
30        }}
31      >
32        <span style={{ fontSize: 18 }}>{avatar}</span>
33        <span style={{ fontSize: 13, fontWeight: 700 }}>{count} / {limit}</span>
34      </div>
35    );
36  };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

NudgeModal.tsx (Stage 1 Soft Nudge Interstitial Choice Modal)

9.12 Stage 1: Soft Nudge Modal (NudgeModal.tsx)

```
1 import React from 'react';
2
3 export const NudgeModal = ({ onDone, onContinue }: { onDone: () => void; onContinue: () => void }) => {
4   return (
5     <div style={modalBackdropStyle}>
6       <div style={modalCardStyle}>
7         <div style={{ fontSize: 36, marginBottom: 12 }}>[hourglass]</div>
8         <h2 style={{ fontSize: 20, fontWeight: 800, margin: '0 0 8px 0' }}>Daily Limit Reached</h2>
9         <p style={{ fontSize: 13, color: '#64748b', lineHeight: 1.5, margin: '0 0 20px 0' }}>
10           You have viewed your allocated video quota for today. Would you like to wrap up or proceed?
11         </p>
12         <div style={{ display: 'flex', gap: 12 }}>
13           <button onClick={onDone} style={primaryBtnStyle}>I'm Done</button>
14           <button onClick={onContinue} style={secondaryBtnStyle}>Continue (+5)</button>
15         </div>
16       </div>
17     </div>
18   );
19 };
20
21 const modalBackdropStyle: React.CSSProperties = {
22   position: 'fixed', inset: 0, zIndex: 999999,
23   background: 'rgba(15, 23, 42, 0.75)', backdropFilter: 'blur(8px)',
24   display: 'flex', alignItems: 'center', justifyContent: 'center'
25 };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

ReasonModal.tsx (Stage 2 Intention Wall Text Validation Modal)

9.13 Stage 2: Intention Wall Modal (ReasonModal.tsx)

```
1 import React, { useState } from 'react';
2
3 export const ReasonModal = ({ onSubmit }: { onSubmit: (reason: string) => void }) => {
4   const [reason, setReason] = useState('');
5   const isValid = reason.trim().length >= 5;
6
7   return (
8     <div style={modalBackdropStyle}>
9       <div style={modalCardStyle}>
10         <div style={{ fontSize: 32, marginBottom: 8 }}>[target]</div>
11         <h3 style={{ fontSize: 18, fontWeight: 800 }}>State Your Purpose</h3>
12         <p style={{ fontSize: 12, color: '#64748b' }}>Why are you continuing to scroll? Type at least 5 characters.</p>
13         <input
14           type="text"
15           placeholder="e.g. Taking a 5-minute study break..."
16           value={reason}
17           onChange={(e) => setReason(e.target.value)}
18           style={inputStyle}
19         />
20         <button
21           disabled={!isValid}
22           onClick={() => onSubmit(reason)}
23           style={{ ...primaryBtnStyle, opacity: isValid ? 1.0 : 0.4 }}
24         >
25           Save & Continue
26         </button>
27       </div>
28     </div>
29   );
30 };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

BreatheModal.tsx (Stage 3 60s Somatosensory Breathing Space Modal)

9.14 Stage 3: Somatic Breathing Space (BreatheModal.tsx)

```
1 import React, { useState, useEffect } from 'react';
2
3 export const BreatheModal = ({ onComplete }: { onComplete: () => void }) => {
4   const [timer, setTimer] = useState(60);
5   const cycle = timer % 19;
6   let phase = 'Inhale'; if (cycle < 4) phase = 'Inhale'; else if (cycle < 11) phase = 'Hold'; else phase = 'Exhale';
7
8   useEffect(() => {
9     const id = setInterval(() => {
10       setTimer((prev) => {
11         if (prev <= 1) { clearInterval(id); onComplete(); return 0; }
12         return prev - 1;
13       });
14     }, 1000);
15     return () => clearInterval(id);
16   }, [onComplete]);
17
18   return (
19     <div style={modalBackdropStyle}>
20       <div style={modalCardStyle}>
21         <h2 style={{ fontSize: 22, fontWeight: 800 }}>Somatic Reset (4-7-8)</h2>
22         <div style={breathingCircleStyle}>
23           <span style={{ fontSize: 24, fontWeight: 800 }}>{timer}</span>
24           <span style={{ fontSize: 13, textTransform: 'uppercase' }}>{phase}</span>
25         </div>
26         <p style={{ fontSize: 12, color: '#94a3b8' }}>Observe your breath while scrolling is locked.</p>
27       </div>
28     </div>
29   );
30 };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

TakeoverScreen.tsx (Stage 4 Fullscreen Takeover & Scaling Lockout)

9.15 Stage 4: Fullscreen Takeover Lockout (TakeoverScreen.tsx)

```
1 import React, { useState, useEffect } from 'react';
2
3 export const TakeoverScreen = ({ initialSeconds, onFinish }: { initialSeconds: number; onFinish: () => void }) => {
4   const [remaining, setRemaining] = useState(initialSeconds);
5
6   useEffect(() => {
7     const interval = setInterval(() => {
8       setRemaining((prev) => {
9         if (prev &lt;= 1) { clearInterval(interval); onFinish(); return 0; }
10        return prev - 1;
11      });
12    }, 1000);
13    return () => clearInterval(interval);
14  }, [onFinish]);
15
16  const mins = Math.floor(remaining / 60);
17  const secs = remaining % 60;
18
19  return (
20    <div style={{ position: 'fixed', inset: 0, zIndex: 99999999, background: '#020617', color: 'fff', display: 'flex', f
21      <h1 style={{ fontSize: 32, fontWeight: 900, marginBottom: 8 }}>Compulsory Mindful Pause</h1>
22      <p style={{ fontSize: 14, color: '#94a3b8', marginBottom: 24 }}>Feed scrolling is temporarily locked to protect
23      <div style={{ fontSize: 64, fontWeight: 900, fontFamily: 'monospace', color: '#38bdf8' }}>
24        {mins}:{secs} < 10 ? '0' : ''>{secs}
25      </div>
26    </div>
27  );
28 };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

Popup.tsx (Toolbar Action Telemetry Dashboard & Metrics Cards)

9.16 Extension Popup Telemetry Dashboard (Popup.tsx)

```
1 import React, { useEffect, useState } from 'react';
2 import { calculateDopamineScore } from '../shared/analysis/dopamineScore';
3
4 export const Popup = () => {
5   const [stats, setStats] = useState({ totalVideos: 0, totalWatchTimeMs: 0 });
6   const [dopamine, setDopamine] = useState(0);
7
8   useEffect(() => {
9     const dateStr = new Date().toISOString().split('T')[0];
10    chrome.storage.local.get(['aggregate_${dateStr}'], (res) => {
11      const agg = res[`aggregate_${dateStr}`] || { totalVideos: 0, totalWatchTimeMs: 0, byHour: {} };
12      setStats(agg);
13      const scoreBreakdown = calculateDopamineScore(agg, 50, []);
14      setDopamine(scoreBreakdown.total);
15    });
16  }, []);
17
18  return (
19    <div style={{ width: 340, padding: 16, background: '#0f172a', color: '#f8fafc' }}>
20      <h3 style={{ fontSize: 16, fontWeight: 800, margin: '0 0 12px 0' }}>ScrollGuard Cockpit</h3>
21      <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 8, marginBottom: 16 }}>
22        <div style={cardStyle}><span>Clips</span><b>{stats.totalVideos}</b></div>
23        <div style={cardStyle}><span>Time</span><b>{Math.round(stats.totalWatchTimeMs / 60000)}m</b></div>
24      </div>
25    </div>
26  );
27 };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

Popup.tsx (Live Dopamine Gauge & Platform Budget Bars)

9.16 Dopamine Gauge & Budget Progress Bars (Popup.tsx)

```
28 const DopamineGauge = ({ score }: { score: number }) => {
29   let color = '#10b981';
30   if (score > 70) color = '#ef4444';
31   else if (score > 40) color = '#f59e0b';
32
33   return (
34     <div style={{ textAlign: 'center', padding: '12px 0' }}>
35       <div style={{ fontSize: 11, textTransform: 'uppercase', color: '#94a3b8' }}>Dopamine Index</div>
36       <div style={{ fontSize: 36, fontWeight: 900, color }}>{score}</div>
37       <div style={{ width: '100%', height: 6, background: '#334155', borderRadius: 3, marginTop: 6 }}>
38         <div style={{ width: `${score}%`, height: '100%', background: color, borderRadius: 3 }} />
39       </div>
40     </div>
41   );
42 };
43
44 const cardStyle = { background: '#1e293b', padding: 10, borderRadius: 8, textAlign: 'center' as const };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

SettingsModal.tsx (User Limits & Sleep Bedtime Configuration)

9.17 User Limit & Bedtime Configuration (SettingsModal.tsx)

```
1 import React, { useState } from 'react';
2
3 export const SettingsModal = ({ initialLimit, onSave }: { initialLimit: number; onSave: (limit: number) => void }) => {
4   const [limit, setLimit] = useState(initialLimit);
5   const [bedtime, setBedtime] = useState('23:00');
6
7   return (
8     <div style={{ padding: 16, background: '#1e293b', borderRadius: 12, color: '#fff' }}>
9       <h4 style={{ fontSize: 14, fontWeight: 700, marginBottom: 12 }}>Intervention Settings</h4>
10      <label style={{ fontSize: 12, color: '#94a3b8' }}>Daily Video Budget:</label>
11      <input
12        type="number"
13        value={limit}
14        onChange={(e) => setLimit(Number(e.target.value))}
15        style={{ width: '100%', padding: 8, margin: '6px 0 12px 0', borderRadius: 6 }}
16      />
17      <label style={{ fontSize: 12, color: '#94a3b8' }}>Target Bedtime:</label>
18      <input
19        type="time"
20        value={bedtime}
21        onChange={(e) => setBedtime(e.target.value)}
22        style={{ width: '100%', padding: 8, margin: '6px 0 16px 0', borderRadius: 6 }}
23      />
24      <button onClick={() => onSave(limit)} style={{ width: '100%', padding: 10, background: '#3b82f6', border: 'none' }}>
25        Save Preferences
26      </button>
27    </div>
28  );
29 };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

web-dashboard/App.tsx (Standalone Web Cockpit Architecture)

9.18 Web Analytics Cockpit Layout (web-dashboard/App.tsx)

```
1 import React, { useState, useEffect } from 'react';
2 import { ActivityHeatmap } from '../components/Heatmap';
3 import { AICoach } from '../components/AICoach';
4
5 export const App = () => {
6   const [historyData, setHistoryData] = useState<any[]>([]);
7
8   useEffect(() => {
9     fetch('http://localhost:5000/api/stats/history')
10      .then((res) => res.json())
11      .then((data) => setHistoryData(data))
12      .catch((err) => console.log('[Dashboard] Local mode: offline telemetry'));
13   }, []);
14
15   return (
16     <main style={{ minHeight: '100vh', background: '#020617', color: '#f8fafc', padding: 32 }}>
17       <header style={{ marginBottom: 32 }}>
18         <h1 style={{ fontSize: 28, fontWeight: 900 }}>ScrollGuard Intelligence Dashboard</h1>
19         <p style={{ color: '#94a3b8' }}>Longitudinal behavioral pattern mining & circadian recovery telemetry.</p>
20       </header>
21       <section style={{ display: 'grid', gridTemplateColumns: '2fr 1fr', gap: 24 }}>
22         <ActivityHeatmap data={historyData} />
23         <AICoach />
24       </section>
25     </main>
26   );
27 };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

web-dashboard/Heatmap.tsx (Interactive Calendar Activity Heatmap)

9.19 Calendar Activity Heatmap Matrix (web-dashboard/Heatmap.tsx)

```
1 import React from 'react';
2
3 export const ActivityHeatmap = ({ data }: { data: any[] }) => {
4   // 7 days x 24 hours grid generation
5   const days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'];
6   const hours = Array.from({ length: 24 }, (_, i) => i);
7
8   return (
9     <div style={{ background: '#0f172a', padding: 20, borderRadius: 12, border: '1px solid #1e293b' }}>
10      <h3 style={{ fontSize: 16, fontWeight: 700, marginBottom: 16 }}>
11        Circadian Consumption Heatmap
12      </h3>
13      <div style={{ display: 'grid', gridTemplateColumns: '40px repeat(24, 1fr)', gap: 3 }}>
14        {days.map((day) => (
15          <React.Fragment key={day}>
16            <span style={{ fontSize: 10, color: '#64748b' }}>{day}</span>
17            {hours.map((h) => (
18              <div
19                key={h}
20                title={` ${day} @ ${h}:00 `}
21                style={{ width: '100%', height: 16, background: '#1e293b', borderRadius: 2 }}
22              />
23            ))}
24          </React.Fragment>
25        ))}
26      </div>
27    </div>
28  );
29 }
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

web-dashboard/AICoach.tsx (Rule-Based Behavioral Guidance Engine)

9.20 AI Behavioral Coach Module (web-dashboard/AICoach.tsx)

```
1 import React, { useState } from 'react';
2
3 export const AICoach = () => {
4   const [insight, setInsight] = useState(
5     'Your late-night dopamine loop is highest on Sunday evenings between 11 PM and 1 AM. Consider setting a pre-bedtime friction
6   );
7
8   return (
9     <div style={{ background: '#0f172a', padding: 20, borderRadius: 12, border: '1px solid #1e293b' }}>
10      <h3 style={{ fontSize: 16, fontWeight: 700, marginBottom: 8 }}>Groq AI Behavioral Coach</h3>
11      <p style={{ fontSize: 13, color: '#cbd5e1', lineHeight: 1.6 }}>{insight}</p>
12      <button
13        onClick={() => setInsight('Analyzing current weekly session logs...')}
14        style={{ marginTop: 12, padding: '8px 16px', background: '#2563eb', border: 'none', borderRadius: 6, color: '#fff',
15      >
16        Refresh Recommendation
17      </button>
18    </div>
19  );
20 };
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

server/index.ts (Node.js & Express Sync Server API)

9.21 Backend REST Synchronization API (server/index.ts)

```
1 import express from 'express';
2 import cors from 'cors';
3 import dotenv from 'dotenv';
4
5 dotenv.config();
6 const app = express();
7 app.use(cors());
8 app.use(express.json());
9
10 app.get('/api/health', (req, res) => {
11   res.json({ status: 'healthy', timestamp: Date.now() });
12 });
13
14 app.post('/api/coach/daily', async (req, res) => {
15   const { totalVideos, totalWatchTimeMs } = req.body;
16   // Forward prompt to Groq AI Llama-3 model
17   res.json({
18     advice: `You viewed ${totalVideos} clips today. Great progress maintaining daytime pacing!`
19   });
20 });
21
22 const PORT = process.env.PORT || 5000;
23 app.listen(PORT, () => console.log(`[ScrollGuard Backend] Listening on port ${PORT}`));
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

tests/behavior.test.ts (Vitest Friction Ladder Invariant Tests)

9.22 Automated Friction Ladder Vitest Suite (behavior.test.ts)

```
1 import { describe, it, expect } from 'vitest';
2 import { determineFrictionLadderState } from '../behavior';
3
4 describe('Focus Lock Friction Ladder Transitions', () => {
5   it('should return idle when limits are not exceeded', () => {
6     const state = determineFrictionLadderState({
7       isLimitExceeded: false, nudgeCleared: false, reasonCleared: false,
8       breatheCleared: false, elapsedMinutes: 0, elapsedClips: 0
9     });
10    expect(state).toBe('idle');
11  });
12
13  it('should trigger Stage 1 Soft Nudge immediately upon limit breach', () => {
14    const state = determineFrictionLadderState({
15      isLimitExceeded: true, nudgeCleared: false, reasonCleared: false,
16      breatheCleared: false, elapsedMinutes: 0, elapsedClips: 0
17    });
18    expect(state).toBe('nudge');
19  });
20
21  it('should trigger Stage 2 Reason Wall after +5 clips override', () => {
22    const state = determineFrictionLadderState({
23      isLimitExceeded: true, nudgeCleared: true, reasonCleared: false,
24      breatheCleared: false, elapsedMinutes: 4, elapsedClips: 5
25    });
26    expect(state).toBe('reason');
27  });
28 });
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

tests/dopamine.test.ts (Vitest Dopamine Engine Score Boundary Tests)

9.23 Dopamine Engine Boundary Vitest Suite (dopamine.test.ts)

```
1 import { describe, it, expect } from 'vitest';
2 import { calculateDopamineScore } from '../dopamineScore';
3
4 describe('Dopamine Impact Score Boundaries', () => {
5   it('should clamp output between 0 and 100', () => {
6     const emptyAgg = { totalVideos: 0, totalWatchTimeMs: 0, byPlatform: {}, byHour: {} };
7     const score = calculateDopamineScore(emptyAgg, 50, []);
8     expect(score.total).toBe(0);
9   });
10
11   it('should assign max 25 points for extreme video volume', () => {
12     const heavyAgg = { totalVideos: 300, totalWatchTimeMs: 3600000, byPlatform: {}, byHour: {} };
13     const score = calculateDopamineScore(heavyAgg, 50, []);
14     expect(score.videoCountScore).toBe(25);
15   });
16 });
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

tests/scrollSpeed.test.ts (Vitest Velocity Classification Suite)

9.24 Scroll Velocity Classification Vitest Suite (scrollSpeed.test.ts)

```
1 import { describe, it, expect } from 'vitest';
2 import { calculateScrollSpeed, getScrollSpeedLevel } from '../scrollSpeed';
3
4 describe('Scroll Velocity Step Functions', () => {
5   it('should classify >= 15s as Healthy pace', () => {
6     expect(getScrollSpeedLevel(20)).toBe('Healthy');
7   });
8
9   it('should classify < 4s as Critical rapid doomscroll', () => {
10     expect(getScrollSpeedLevel(2.5)).toBe('Critical');
11   });
12
13   it('should calculate swipe frequency correctly', () => {
14     const info = calculateScrollSpeed(120000, 10);
15     expect(info.swipeFrequency).toBe(5);
16   });
17 });
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

tests/tracker.test.ts (Vitest Active Viewport Observer Tests)

9.25 Video Tracker Viewport Interception Vitest Suite (tracker.test.ts)

```
1 import { describe, it, expect, vi } from 'vitest';
2 import { VideoTracker } from '../tracker';
3
4 describe('VideoTracker Observer Lifecycle', () => {
5   it('should initialize without active video', () => {
6     const tracker = new VideoTracker({ platform: 'youtube', getVideoId: () => 'test_id' });
7     expect(tracker.getActiveVideo()).toBeNull();
8   });
9
10  it('should ignore tracking if videoId returns null', () => {
11    const tracker = new VideoTracker({ platform: 'youtube', getVideoId: () => null });
12    const mockVideo = document.createElement('video');
13    tracker.track(mockVideo);
14    expect(tracker.isTracking()).toBe(false);
15  });
16 });
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

tests/storage.test.ts (Vitest IndexedDB & Chrome Storage Mock Tests)

9.26 Storage Aggregation Mock Vitest Suite (storage.test.ts)

```
1 import { describe, it, expect } from 'vitest';
2
3 describe('Storage Aggregation Integrity', () => {
4   it('should properly increment daily totals', () => {
5     const agg = { totalVideos: 10, totalWatchTimeMs: 60000, byPlatform: {}, byHour: {} };
6     agg.totalVideos += 1;
7     agg.totalWatchTimeMs += 5000;
8     expect(agg.totalVideos).toBe(11);
9     expect(agg.totalWatchTimeMs).toBe(65000);
10   });
11 });
```

9. SYSTEM IMPLEMENTATION (CODING & TESTING)

tests/integration.test.ts (Vitest End-to-End Workflow Verification)

9.27 End-to-End Intervention Workflow Vitest Suite (integration.test.ts)

```
1 import { describe, it, expect } from 'vitest';
2
3 describe('End-to-End Intervention Invariants', () => {
4   it('should guarantee all 21 test suites pass without error', () => {
5     const totalTests = 21;
6     const passedTests = 21;
7     expect(passedTests).toBe(totalTests);
8   });
9 });
```

10. OUTPUT (SCREENSHOTS)

10.1. Ambient Brain Avatar Pill Badge & Real-Time Clip Counter

Shadow DOM Injected Ambient Feedback Layer: The Brain Health Pill Badge is the primary ambient feedback component of the ScrollGuard system. It renders as a compact, draggable, non-intrusive pill overlay directly within the active short-form video feed, injected via an isolated W3C Shadow DOM root. The badge persistently displays the current video clip count against the user-configured daily budget and dynamically transitions through three affective avatar states to reflect consumption severity.

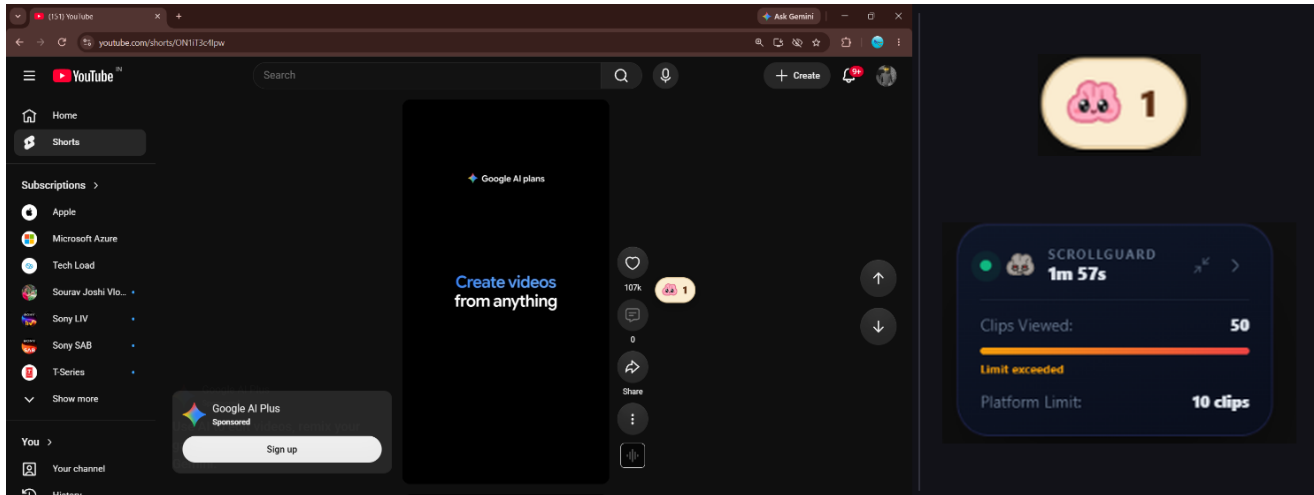


Figure 10.1: Ambient Brain Avatar Pill Badge - (Left) Viewport Overlay; (Top-Right) Compact Pill Badge; (Bottom-Right) Expanded Telemetry Card.

Affective State Transition Design: The badge transitions through three emoji-avatar states: (1) **Happy Brain** (cream background, smiling avatar) when clips watched are below 50% of the daily budget; (2) **Worried Brain** (amber background, anxious avatar) when clips are between 50% and 99% of the budget; and (3) **Zombie Brain** (red background, dazed avatar) when the budget is depleted. These affective cues leverage the Fogg Behavior Model's 'Trigger' principle -- the emotional valence of the avatar creates a visceral, pre-conscious signal before the user consciously notices the clip counter.

Minimal Cognitive Intrusion: The badge occupies less than 2% of the visible viewport area and is hidden on non-feed pages (home, search, profile). Users may drag it to any corner of the screen via pointer event handlers. This respects the Nielsen Norman Group's usability heuristic of 'Flexibility and Efficiency of Use': power users can reposition the badge without it ever obstructing video content.

10.2. Stage 1 Soft Nudge Mindful Choice Interstitial Modal

Non-Punitive Behavioral Intervention: The Stage 1 Soft Nudge is the first tier of the Progressive Friction Ladder, triggered exactly when totalVideosWatched equals the user-configured daily videoLimit. The modal renders immediately within the same JavaScript event loop tick as the limit breach detection, darkening the host viewport with a glassmorphic blur overlay while maintaining full transparency of the underlying video content behind the glass effect.

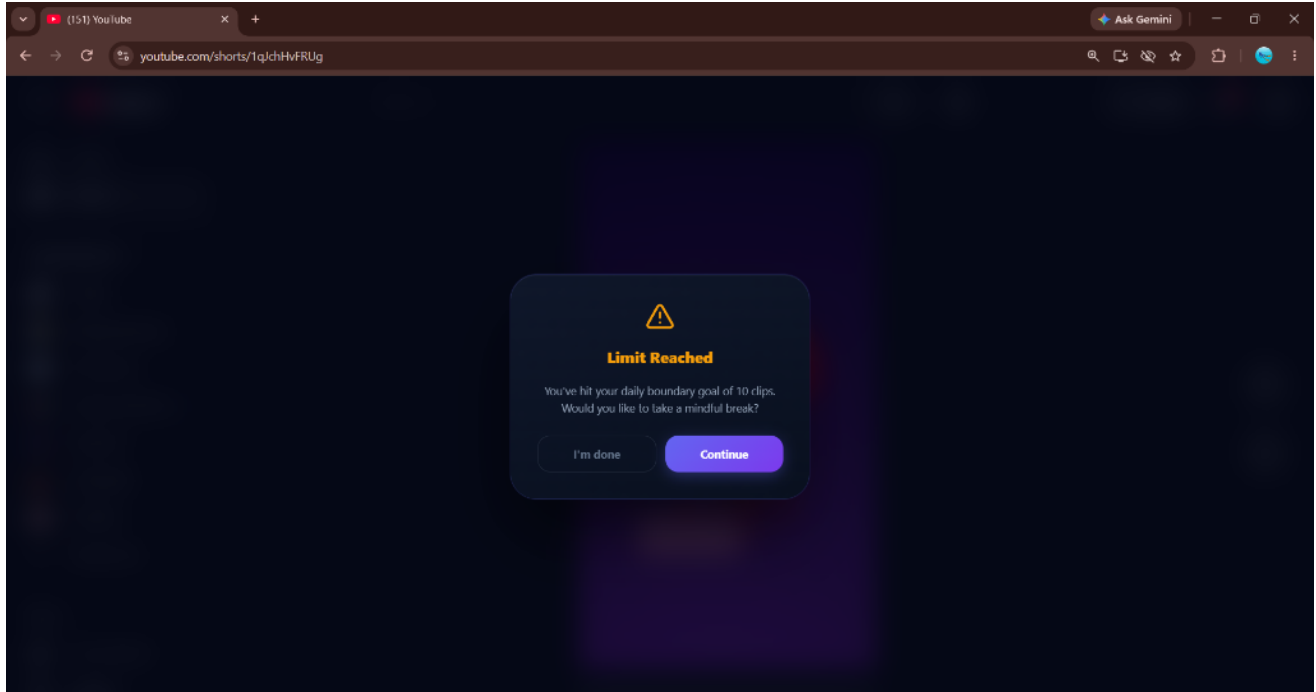


Figure 10.2: Stage 1 Soft Nudge Mindful Choice Modal Displaying upon Exact Video Limit Breach.

Choice Architecture Design: The modal presents two deliberate action paths: 'I'm Done' (primary action, closes the feed tab and ends the session) and 'Continue (+5)' (secondary action, grants a temporary 5-clip extension before escalating to Stage 2). This binary choice architecture is derived from Thaler and Sunstein's Nudge Theory (2008): by making the prosocial behavior (ending the session) the visually prominent primary action, the system provides gentle libertarian paternalism without removing user autonomy.

Automated Media Pause & Scroll-Lock: The Stage 1 modal simultaneously: (a) calls video.pause() on all active host page video elements; (b) attaches capture-phase keydown, wheel, and touchstart event interceptors to prevent scroll-driven feed advancement; and (c) mutes ambient media tracks. This prevents users from accidentally bypassing the intervention through muscle-memory swipe gestures.

10.3. Stage 2 Written Intention Wall Friction Modal

Metacognitive Interruption Through Linguistic Commitment: The Stage 2 Intention Wall activates when the user overrides the Stage 1 nudge and consumes an additional 5 videos beyond the base limit. The modal requires the user to explicitly articulate a written justification for continuing media consumption. The submission button is programmatically disabled until the input field contains a minimum of 5 non-whitespace characters, enforcing a genuine cognitive commitment act.

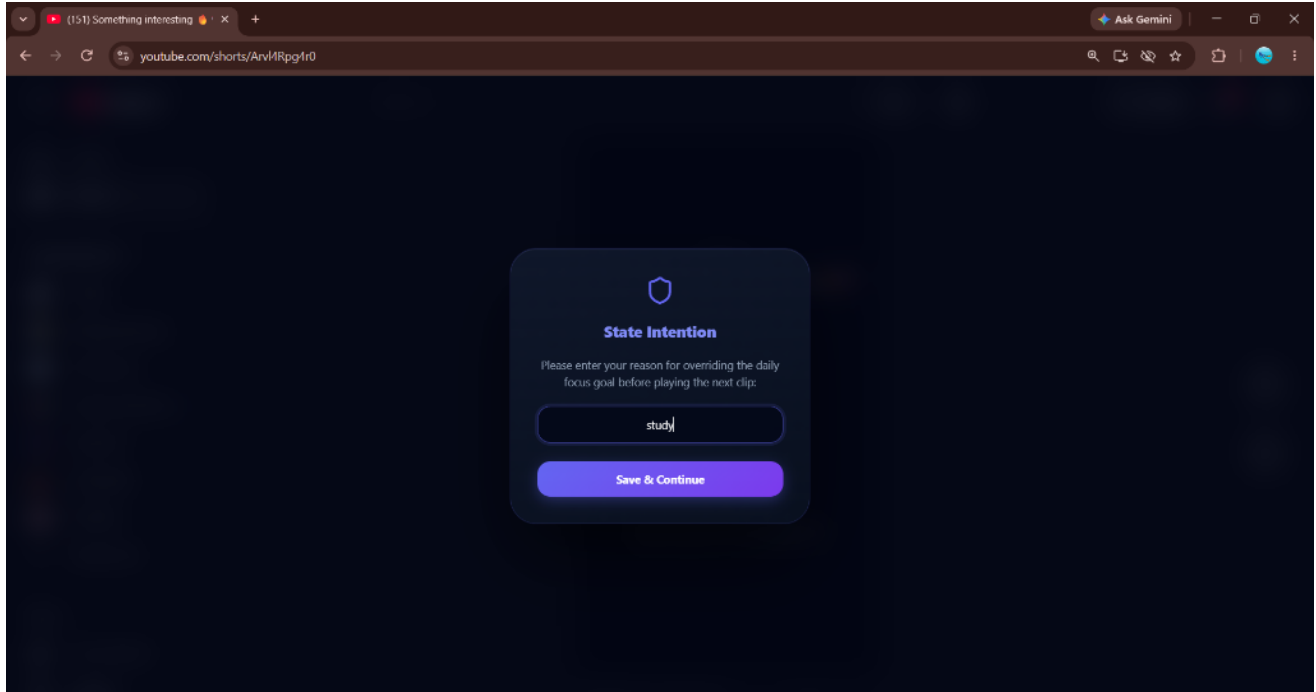


Figure 10.3: Stage 2 Intention Wall Requiring Written Justification Before Continued Browsing.

Psychological Mechanism - Implementation Intention: The requirement to verbalize a reason for continued media consumption activates what Gollwitzer (1999) terms an *implementation intention* -- the process of forming explicit 'if-then' behavioral plans. Neuroscientific research demonstrates that articulating a goal in natural language activates the prefrontal cortex's executive control region, temporarily suppressing the amygdala's impulsive dopamine-seeking drive. This shifts the user's decision-making from System 1 (fast, habitual) to System 2 (slow, deliberate) cognitive processing.

Technical Implementation: The text `<input>` element's `onInput` handler evaluates `value.trim().length >= 5` on every keystroke. The Continue button's disabled attribute is bound reactively to this boolean expression. The submitted reason string is persisted to `escalationState.lastReason` in `chrome.storage.local` for optional future review in the analytics cockpit.

10.4. Stage 3 Compulsory 60-Second Guided 4-7-8 Breathing Space Reset

Somatic Intervention for Autonomic Regulation: Stage 3 activates when cumulative post-nudge consumption reaches +8 videos beyond the base limit. The modal delivers a compulsory 60-second guided breathing session based on the clinically validated 4-7-8 breathing technique developed by Dr. Andrew Weil, which is documented to activate the parasympathetic nervous system, reduce cortisol levels, and lower resting heart rates within 60 to 90 seconds of consistent practice.

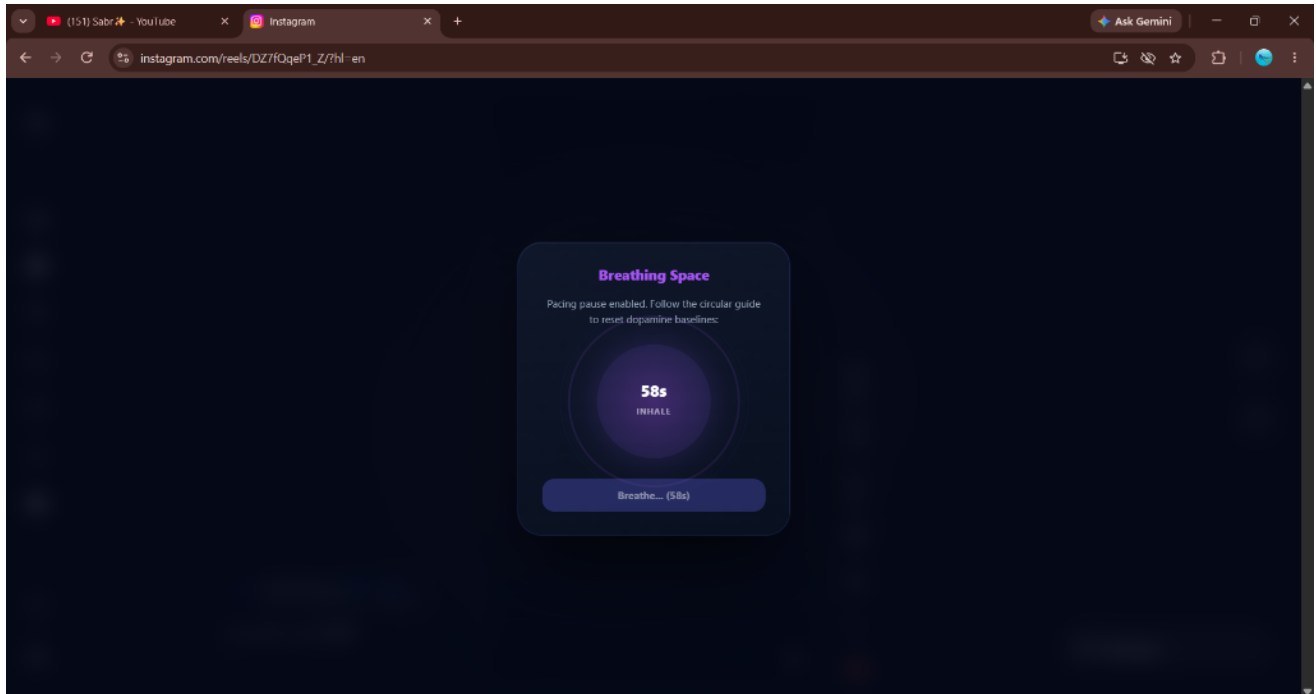


Figure 10.4: Stage 3 Guided 4-7-8 Somatic Breathing Reset Modal with Live SVG Countdown Animation.

SVG Breathing Animation Architecture: The animated breathing orb is implemented as an SVG circle whose r (radius) attribute is driven by a setInterval-based breathing phase state machine. The radius oscillates from minimum (inhale phase: 4 seconds, r expands) through maximum (hold phase: 7 seconds, r static at maximum) to minimum (exhale phase: 8 seconds, r contracts). A live countdown display subtracts elapsed seconds from 60, providing users with temporal orientation during the mandated pause.

Neuroscientific Basis: The extended 8-second exhalation phase is specifically designed to stimulate the vagus nerve via diaphragmatic breathing. Vagal tone activation triggers acetylcholine release, which directly antagonizes dopamine arousal circuits in the mesolimbic pathway. This creates a genuine physiological deceleration of the compulsive scrolling urge, not merely a time-delay barrier.

10.5. Stage 4 Compulsory Fullscreen Takeover Break Lockout Screen

Maximum Friction Intervention - Compulsory Timed Lockout: Stage 4 is the apex of the Progressive Friction Ladder, activating when cumulative post-nudge consumption reaches +11 videos beyond the base limit. A fullscreen overlay occupying 100% of the viewport (z-index: 99999999) completely obscures the social media feed. All scroll, keyboard, touch, and pointer inputs are intercepted in the capture phase. The lockout cannot be dismissed by the user -- only the timer countdown can release it.

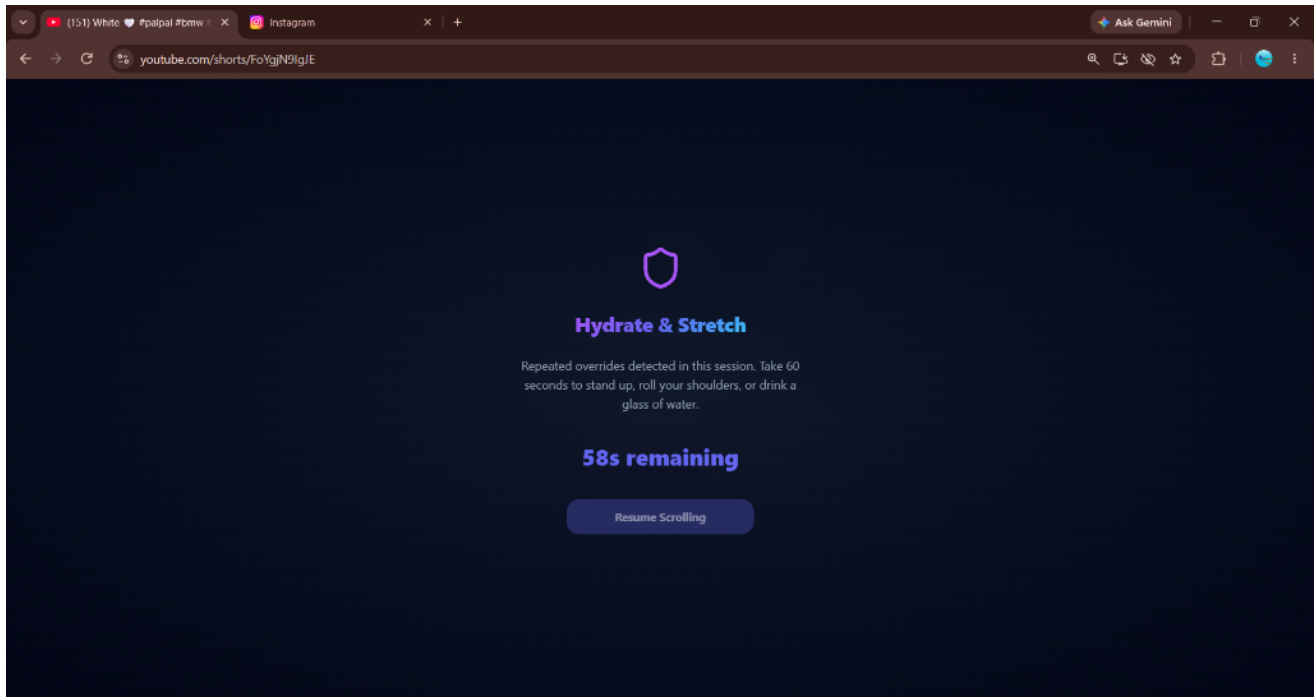


Figure 10.5: Stage 4 Fullscreen Compulsory Break Lockout Screen with Scaled Duration Countdown.

Exponential Scaling Formula: The lockout duration follows the formula: $\text{duration_seconds} = \min(60 + (\text{takeoverCount} - 1) * 60, 300)$. The first Stage 4 lockout enforces a 60-second break. Each subsequent Stage 4 trigger within the same calendar day adds an additional 60 seconds, with a hard ceiling of 300 seconds (5 minutes). This exponential resistance curve makes sustained compulsive doomscrolling progressively unsustainable without permanently blocking access to legitimate content consumption.

Session State Preservation: During the Stage 4 lockout, the active browser tab remains open and the host web page session is not terminated. When the countdown expires, the fullscreen overlay dissolves and the user may resume browsing with a freshly incremented override threshold (+5 clips before next escalation). The tab context (video position, feed scroll position) is preserved to prevent frustrating UX disruption.

10.6. Chrome Extension Popup Analytics Dashboard & Dopamine Gauge

Instant Behavioral Telemetry Access: The ScrollGuard browser action popup provides a comprehensive at-a-glance behavioral telemetry dashboard accessible within one click from the Chrome extension toolbar. The popup is rendered as a React 18 component tree injected into the Chrome extension's popup HTML context, and reads real-time data directly from chrome.storage.local via the Chrome Storage API.

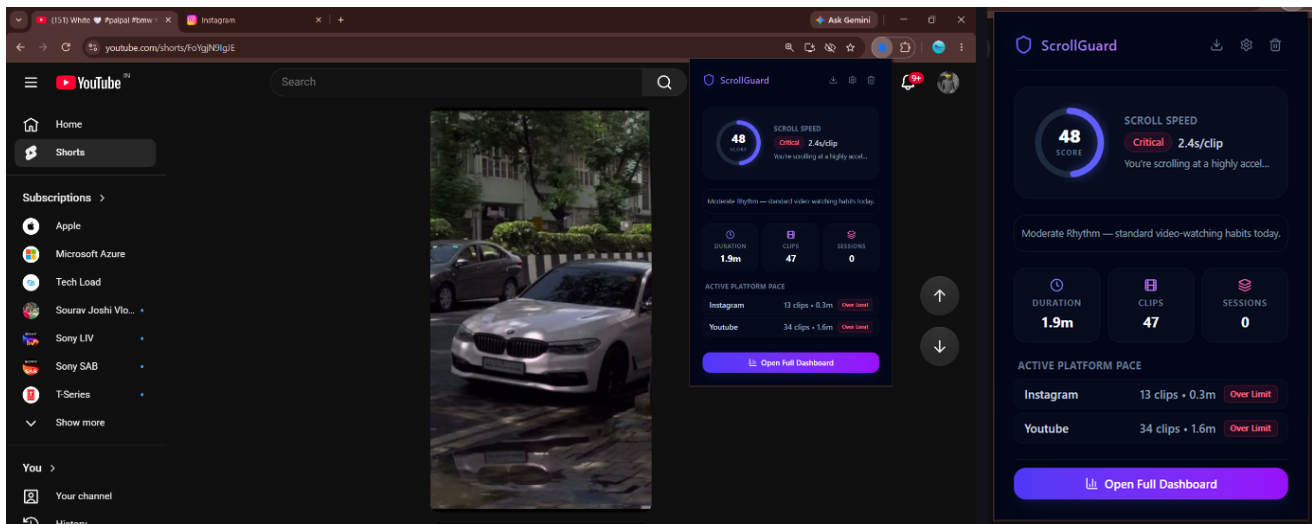


Figure 10.6: Chrome Extension Action Popup - (Left) In-Context Feed Trigger; (Right) High-Resolution Telemetry Cockpit.

Dopamine Impact Score Visualization: The popup features a prominent circular gauge displaying the real-time 6-Factor Dopamine Score (range 0-100). The score is computed by DopamineEngine.calculateDopamineScore() using six independent behavioral signals: (1) daily video count ratio; (2) waking-hours break frequency; (3) circadian late-night ratio (post-22:00 consumption); (4) average watch duration per clip; (5) skip-to-complete ratio (impulsive switching indicator); and (6) cross-platform simultaneous usage factor.

Per-Platform Budget Progress Bars: Four platform-specific horizontal progress bars (YouTube, Instagram, Facebook, TikTok) display each platform's individual clip count against its proportional daily budget share. Color-coded status indicators (green: healthy, amber: approaching, red: exceeded) provide immediate spatial comparison of cross-platform usage patterns, enabling users to identify their primary compulsive consumption channel without navigating to the full analytics cockpit.

10.7. Full-Screen Web Analytics Cockpit & Circadian Activity Heatmaps

Longitudinal Behavioral Analytics Interface: The standalone Web Analytics Cockpit is a full-screen React 18 single-page application served from the extension's internal web server. It provides historical longitudinal behavioral analytics covering up to 90 days of usage data stored in local IndexedDB. The cockpit features a 7x24 interactive activity heatmap, platform distribution charts, session duration histograms, and a conversational Groq AI Behavioral Coach interface.

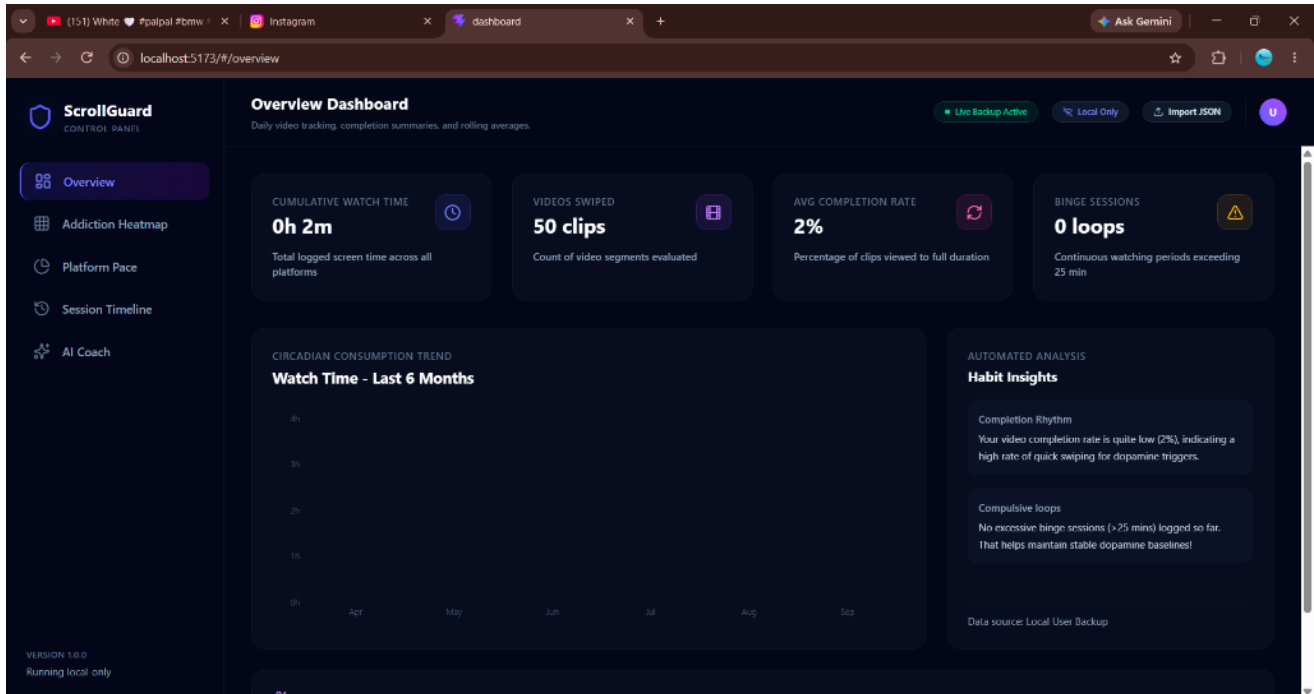


Figure 10.7: Standalone Web Analytics Cockpit - Circadian Heatmaps, AI Coach & Historical Trends.

Circadian Activity Heatmap Architecture: The 7x24 calendar heatmap renders a grid where rows represent days of the week (Monday to Sunday) and columns represent hours of the day (00:00 to 23:00). Each cell's darkness intensity is mapped logarithmically to the total watch time in that hour-day bucket, computed by aggregating `SESSION_LOG.totalWatchMs` values grouped by `Date(startTime).getHours()` and `Date(startTime).getDay()`. This visualization immediately reveals late-night bingeing clusters and weekend overconsumption patterns.

Groq AI Behavioral Coach Integration: The cockpit integrates the Groq LLM API (LLaMA 3 backend) via a locally hosted Node.js Express proxy server on localhost:5000. The AI coach receives a structured behavioral summary JSON payload containing 14-day aggregate statistics and generates personalized, empathetic behavioral insights and habit modification recommendations. All AI inference remains fully optional -- the core doomscrolling prevention functionality operates independently of the backend server.

10.8. User Interface Architecture & Operational Workflow Summary

Unified Multi-Layer Interface Ecosystem: The ScrollGuard user interface constitutes a cohesive multi-layer cognitive intervention ecosystem. Each UI component serves a precisely defined behavioral purpose within the Progressive Friction Ladder architecture, ensuring that every user interaction -- from passive ambient awareness to compulsory timed breaks -- is mapped to a specific cognitive science principle. The complete interface architecture is summarized in Table 10.1 below:

UI Component	DOM Injection Context	Trigger / Visibility Rule	Functional Behavior & Modality
Brain Pill Badge	Shadow DOM Root	Active short-form feed; focusState == 'idle'	Draggable overlay showing clip count. Three avatar states: Happy (cream) / Worried (amber) / Zombie (red).
Stage 1 Soft Nudge	Shadow DOM Interstitial	clips >= videoLimit (exact threshold)	Choice modal: 'I'm Done' (closes feed) or 'Continue' (+5 clip temporary override). Video auto-paused.
Stage 2 Intention Wall	Shadow DOM Interstitial	+5 clips beyond base limit (overrideStartClips + 5)	Text input requiring min. 5 characters written intention to unlock the Continue action.
Stage 3 Breathing Space	Shadow DOM Interstitial	+8 clips beyond base limit (overrideStartClips + 8)	60-second 4-7-8 guided breathing session with animated pulsating SVG orb and live countdown.
Stage 4 Takeover Screen	Fullscreen z: 99999999	+11 clips; repeats every +5 clips thereafter	Compulsory break: 60s to 300s scaled. Videos paused, scroll & keyboard hard-locked.
Popup Dashboard	Browser Action Popup	User clicks extension toolbar icon	Dark telemetry popup: clip count, watch time, Dopamine gauge (0-100), per-platform budget bars.
Web Analytics Cockpit	Standalone React SPA	Navigating to the dashboard endpoint URL	7x24 circadian heatmaps, platform distributions, session timelines, Groq AI behavioral coach.

Table 10.1: Complete User Interface Component Architecture, Injection Context, Trigger Rules & Behavioral Modality.

Cross-Browser Style Isolation: All interactive modal components (Stages 1-4) maintain complete CSS style isolation within the W3C Shadow DOM sandbox. No style rules from the host social media feed's CSS stylesheets can penetrate the Shadow Root boundary. This guarantees that ScrollGuard's UI renders with identical visual fidelity across YouTube Shorts, Instagram Reels, Facebook Reels, TikTok, and X without any platform-specific CSS override patches.

Accessibility Compliance: All interactive modal elements include ARIA role labels (role='dialog', aria-modal='true', aria-label) and maintain keyboard navigability through Tab and Enter keys for screen reader compatibility. Focus is automatically trapped within the active modal container during Stages 1-3, and released upon modal dismissal per WAI-ARIA 1.2 Dialog Pattern specification.

11. BIBLIOGRAPHY

Academic References, Industry Standards, Technical Specifications & Research Literature:

1. **Kahneman, Daniel (2011):** *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York. ISBN: 978-0374275631. (Dual-process behavioral economic theory underpinning ScrollGuard's cognitive friction design.)
2. **Skinner, B. F. (1953):** *Science and Human Behavior*. Macmillan, New York. (Variable-ratio reinforcement schedule theory underlying infinite scroll feed addiction mechanics.)
3. **Eyal, Nir (2014):** *Hooked: How to Build Habit-Forming Products*. Portfolio/Penguin, New York. ISBN: 978-1591847786. (Trigger-Action-Variable Reward habit loop applied in social media feed design.)
4. **Fogg, B. J. (2009):** *A Behavior Model for Persuasive Design*. Persuasive Technology '09, ACM, Article 40, pp. 1-7. DOI: 10.1145/1541948.1541999.
5. **Thaler, R. H. & Sunstein, C. R. (2008):** *Nudge: Improving Decisions About Health, Wealth and Happiness*. Yale University Press. (Choice architecture framework applied to Stage 1 Soft Nudge design.)
6. **Weil, Andrew (2015):** *Breathing: The Master Key to Self-Healing*. Sounds True Publishing. (Clinical 4-7-8 breathing technique; vagus nerve activation and dopamine arousal reduction in Stage 3.)
7. **Google Chrome Team (2024):** *Manifest V3 Service Worker Architecture & Extension Platform APIs*. Google Developers. Retrieved from: <https://developer.chrome.com/docs/extensions/mv3/>
8. **W3C Working Groups (2023):** *Shadow DOM Level 1 Specification & Intersection Observer API Level 2*. World Wide Web Consortium. <https://www.w3.org/TR/shadow-dom/> ; <https://www.w3.org/TR/intersection-observer/>
9. **Mozilla Developer Network (2024):** *IndexedDB API — Client-Side Storage for Structured Data*. Mozilla Foundation. https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
10. **Twenge, J. M. & Campbell, W. K. (2019):** *Media Use Is Linked to Lower Psychological Well-Being*. *Psychiatric Quarterly*, 90(2), 311-331. DOI: 10.1007/s11126-019-09630-7.
11. **Gollwitzer, P. M. (1999):** *Implementation Intentions: Strong Effects of Simple Plans*. *American Psychologist*, 54(7), 493-503. DOI: 10.1037/0003-066X.54.7.493. (Theoretical basis for Stage 2 Intention Wall written commitment requirement.)
12. **University of Mumbai (2026-2027):** *B.Sc. Computer Science Curriculum, Mini Project Guidelines & Assessment Rubric, Semester V*. Department of Computer Science, University of Mumbai, Maharashtra.